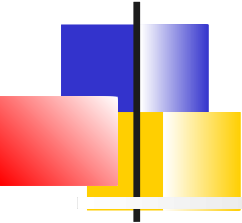


Quarkus Reactive

David THIBAU – 2026

david.thibau@gmail.com



Agenda

Rappels Quarkus

- Positionnement
- Quarkus vs SpringBoot
- Développer avec Quarkus
- Principales Annotations
- Configuration et profils

Programmation réactive

- Problème de la concurrence
- Le modèle réactif
- L'intérêt de la back-pressure
- Reactive Streams et API Flow
- Virtual Threads
- Modèle d'exécution Vert.x
- @Blocking @NonBlocking

Mutiny

- Concepts de base
- Composition et Gestion d'erreurs
- Concepts avancés
- Tests

Persistance réactive

- Hibernate et Panache
- NoSQL

Quarkus REST

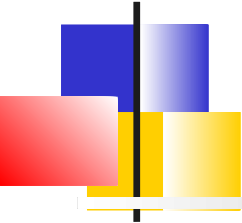
- Introduction
- Détection automatique et annotations
- Bean Validation, Sérialisation
- SSE

Couche Service

- Composition de service
- Rest Client
- Fault Tolerance

Messaging

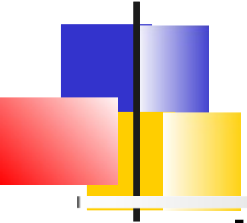
Smallrye Reactive Messaging
Connecteur Kafka
Ack, Erreurs, Stratégies de commit
Tests



Rappels Quarkus

Positionnement

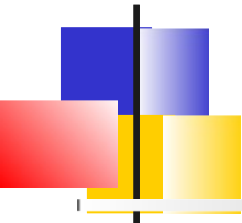
Quarkus vs SpringBoot
Développer avec Quarkus
Principales annotations
Configuration et profils



Le constat Quarkus

Les stacks Java traditionnelles ont été conçues pour les applications monolithiques avec de longs temps de démarrage et de grandes exigences de mémoire

A l'époque le cloud, les conteneurs et Kubernetes n'existaient pas !



La réponse de Quarkus

Quarkus veut fournir un framework Java **Cloud native** pour GraalVM et HotSpot.

Objectifs :

- Faire de Java la plate-forme leader dans les environnements Kubernetes et serverless
- Offrir un framework capable de traiter le plus large éventail d'architectures d'applications distribuées.

Quarkus est full OpenSource (Apache License 2.0)



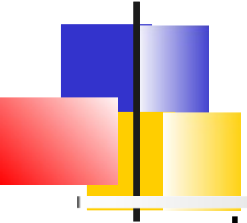
Caractéristiques de Quarkus

Offre une expérience de développement fluide grâce à une combinaison d'outils, de bibliothèques, d'extensions, etc.

Faciliter le déploiement vers Kubernetes, favoriser Dev/Prod parity

Sélection des meilleures librairies Java

Scalabilité via programmation réactive ou impérative et VirtualThread



Build-time

L'idée centrale de Quarkus est de faire durant la compilation ce que les autres frameworks font à l'exécution

- Analyse de la configuration, Scan classpath, Chargement dynamique et Instanciation, etc.

A la compilation, Quarkus prépare et initialise tous les composants utilisés par votre application.

=> Utilisation optimisée de la mémoire et temps de démarrage super-sonique

Quarkus réduit l'utilisation de la réflexion, Il remplace les appels réfléchitifs par des invocations classiques

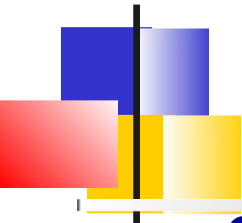
En tant que framework IoC, le graphe d'injection est construit à la compilation



Modèles de programmation

Quarkus fournit du support pour les modèles de programmation modernes

- API Synchrones : RestFul avec JAX-RS, GraphQL, MicroProfile Rest Client Modèle réactif avec Vert.x, Virtual Thread avec Loom
- Persistance : Panache, JPA, NoSQL
- Messaging et Architecture Event-driven pour Kafka, RabbitMQ ou AMQP
- ServerLess / FaaS (Functions As A Service) avec *Funqy* permettant d'être indépendant du fournisseur d'infrastructure ou avec des extensions dédiées (Amazon, Google, ...)



Expérience développeur

Code live : les modifications de code sont automatiquement reflétées dans votre application en cours d'exécution.

Configuration unifiée : Un unique fichier de configuration

Simplicité : Quarkus se concentre sur la manière la plus simple et la plus utile d'utiliser une fonctionnalité donnée

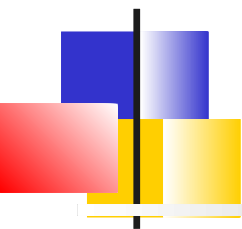
Dev UI : Une interface utilisateur pour les développeurs. Visualiser et configurer les extensions, les beans, accéder aux traces et suite de tests

Dev Services : Intégration automatique avec les services de support tels que les bases de données, les serveurs d'identité, etc. grâce au démarrage de conteneurs.

Test en continu : A chaque changement de code, les tests dépendants sont ré-exécutés

CLI : Un outil pour gérer les extensions et exécuter des commandes de build

Développement Remote : Code live dans un environnement Kubernetes.



Rappels Quarkus

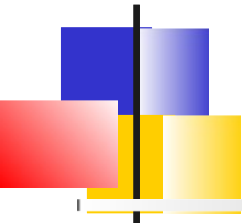
Positionnement

Quarkus vs SpringBoot

Développer avec Quarkus

Principales annotations

Configuration et profils

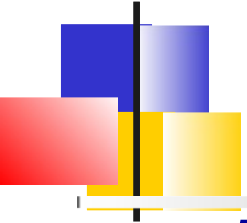


Standards

A la différence de Spring, Quarkus favorise les « standards »

- Standard Eclipse MicroProfile
- Contexts & Dependency Injection (CDI)
- Jakarta RESTful Web Services (JAX-RS)
- Java Persistence API (JPA)
- Java Transaction API (JTA)

Il s'appuie sur certains frameworks cœur :
Eclipse Vert.x, Apache Camel et Hibernate

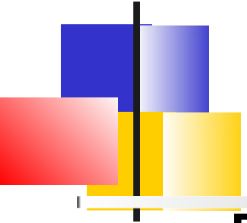


Eclipse MicroProfile

Eclipse MicroProfile est une
spécification définissant les APIs
nécessaires dans une architecture
micro-services

La spécification fait partie de Jakarta EE.

Quarkus est une des implémentations de
ce standard.



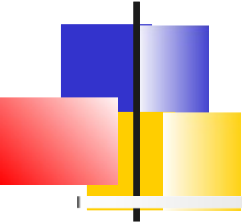
Modèle de threads

Depuis le départ, Quarkus unifie le réactif et l'impératif.

Dans les dernières versions, Java 21, il autorise également l'utilisation des VirtualThreads

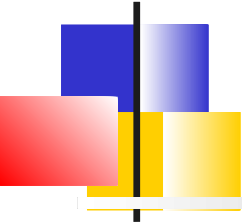
On peut mixer dans le même projet ces différents modes

Spring supporte également ces 3 modes.



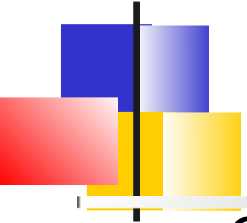
Similitudes

- **Démarrage** : Spring Initializer $\Leftrightarrow$ code.quarkus.io.
- **Librairies** : Starter SpringBoot, $\Leftrightarrow$ Extensions Quarkus
- **Jar Hell** : Contrôle des versions des dépendances : Idem via BOM Maven
- **IoC et DI** : Les applications Quarkus et Spring sont constituées de beans métier ou d'intégration. Le framework a le pouvoir
- **AutoConfiguration** : Quarkus avec les DevServices va encore plus loin en proposant des images Docker des services d'appui
- **Config unique** : 1 seul fichier de configuration, notion de profils de configuration
- **Langages et IDEs** : Idem (Java, Kotlin, vscode, IntelliJ, Eclipse)



Rappels Quarkus

Positionnement
Quarkus vs SpringBoot
Développer avec Quarkus
Principales annotations
Configuration et profils



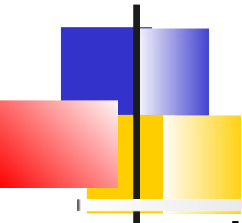
Introduction

Quarkus fournit une chaîne d'outils permettant aux développeurs le « live reload ».

- **Quarkus CLI**
- Equivalence : **plugins Maven** ou **Gradle**

De plus, des plugins et des extensions existent pour les IDEs.

- *VSCode*
- Eclipse
- IntelliJ Version Ultimate

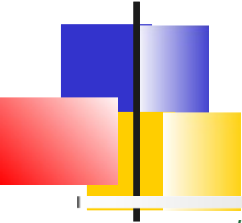


Commandes quarkus-cli

Usage :

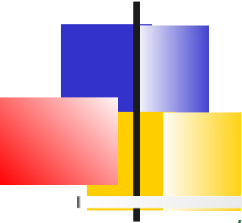
quarkus [-ehv] [--verbose] [-D=<key=value>]...
[COMMAND]

- **create** : Créer un projet, un cli, une extension
- **build** : Construire le projet
- **dev** : Exécuter le projet en mode dev (Live Coding)
- **extension** : Gérer les extensions
- **registry** : Gérer les registres d'extension



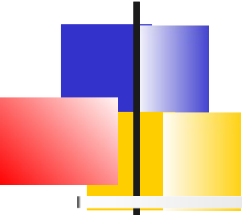
Exemples

```
# Création d'un projet delivery-service # Avec l'extension rest
quarkus create app org.formation:delivery-service -- extension=quarkus-rest
# Démarrage du projet en mode dev cd delivery-service
quarkus dev
# Lister les extensions installables pour le projet
quarkus ext ls -i
# Ajouter un extension
quarkus ext add quarkus-kubernetes quarkus-smallrye-health
```



Exemples Maven

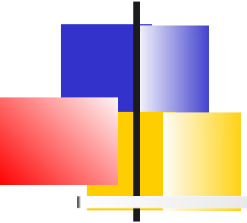
```
# Création d'un projet delivery-service
mvn io.quarkus.platform:quarkus-maven-plugin:2.8.1.Final:create \
    -DprojectGroupId=org.formation \
    -DprojectArtifactId=delivery-service
# Démarrage du projet en mode dev cd delivery-service
./mvnw quarkus:dev
# Lister les extensions installables pour le projet
./mvnw quarkus:list-extensions
# Ajouter un extension
./mvnw quarkus:add-extension -Dextensions="hibernate-validator"
```



pom.xml

Le *pom.xml* d'un projet Quarkus contient :

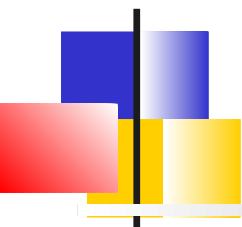
- Des propriétés de version. (en particulier celle de Java et de quarkus)
- Une référence à un BOM permettant de gérer les versions de toutes les dépendances
- Des dépendances sur des extensions
- La configuration des plugins (quarkus, maven-compiler, surefire-plugin)
- Un profil natif pour la génération d'un exécutable natif



Mode développement

Le mode *dev* permet un déploiement à chaud avec compilation en arrière-plan lors de l'actualisation du navigateur

- Sur un reload, si des modifications sont détectées, les fichiers Java sont compilés et l'application est redéployée,
- S'il y a des problèmes avec la compilation ou le déploiement, une page d'erreur vous en informera.



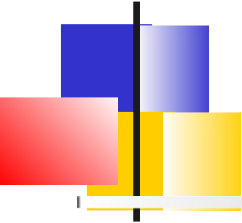
DevUI

DevUI est un outil web permettant d'obtenir des informations sur l'application exécutée en mode *Dev* et éventuellement changer son état.

Son interface dépend des extensions utilisées

Voici quelques exemple de fonctionnalités :

- Lister les clés/valeurs de configuration
- Lister tous les beans CDI et leurs caractéristiques
- Lister les beans inutilisés lors du build
- Lister les entités JPA et leur mapping vers les tables BD
- Les tâches schedulées
- Les rapports de tests
- ...



IDEs

Une fois le projet Maven ou Gradle généré, on peut l'importer dans son IDE

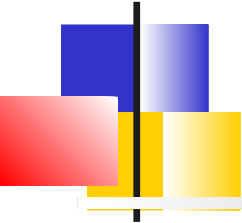
Des plugins Quarkus sont disponibles pour les différents IDEs.

Ils apportent :

- Des assistants de création de projet
- Une gestion graphique des extensions
- Éditeur de propriétés avec complétion

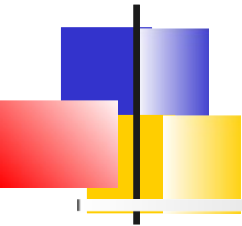
Tableau comparatif disponible ici :

<https://quarkus.io/guides/ide-tooling>



Démarrage dans l'IDE

- Afin de démarrer une application Quarkus dans l'IDE, on s'appuie sur les plugins.
- Le démarrage dans l'IDE permet l'utilisation du debugger de l'IDE
- Par exemple pour Eclipse avec le plugin Quarkus
 - *Run → Run Configurations ...*
 - *QuarkusApplication → New Configuration*
 - Sélectionner le projet et éventuellement le profil
- Pour VSCode et l'extension Quarkus de RedHat
 - Commande : Quarkus: Debug current Quarkus project.

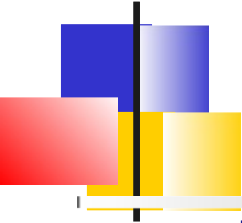


Gestion des traces

En interne, Quarkus utilise *JBoss Log Manager* et la façade *JBoss Logging*.

- On peut alors utiliser directement la façade *JBoss Logging* ou les frameworks supportés (*java.util.logging*, *SLF4J*, *Apache Commons Logging*)

Toute la configuration peut être effectuée dans *application.properties*.

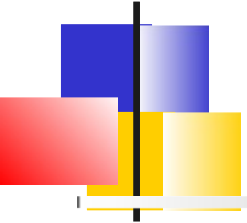


JBoss Logging

```
import org.jboss.logging.Logger;
...
@Path("/hello")
public class ExampleResource {

    private static final Logger LOG =
        Logger.getLogger(ExampleResource.class);

    @GET
    @Produces(MediaType.TEXT_PLAIN) public String hello() {
        LOG.info("Hello");
        return "hello";
    }
}
```

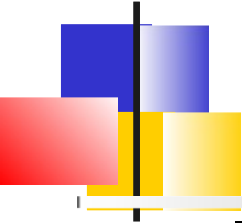


Logging simplifié

Au lieu de déclarer un champ *Logger*, on peut utiliser l'API de logging simplifiée :

```
import io.quarkus.logging.Log;
```

```
class MyService {  
    public void doSomething()  
    {  
        Log.info("Simple!");  
    }  
}
```



Injection de Logger

Il est également possible de s'injecter une instance de *org.jboss.logging.Logger*

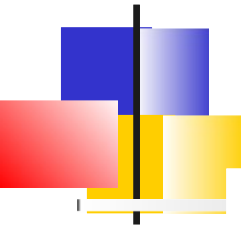
```
import org.jboss.logging.Logger;

@ApplicationScoped
class SimpleBean {

    @Inject
    Logger log;

    @LoggerName("foo")
    Logger fooLog;

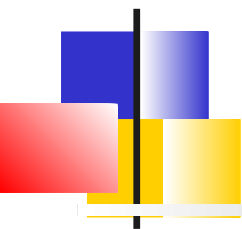
    public void ping() {
        log.info("Simple!");
        fooLog.info("Goes to _foo_ logger!");
    }
}
```



Niveaux de logs

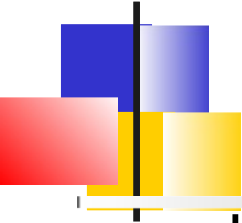
Les niveaux de logs utilisés par Quarkus sont :

- OFF : Désactive le logging.
- FATAL
- ERROR
- WARN
- INFO
- DEBUG
- TRACE
- ALL (Permet d'inclure tous les messages, même les niveaux personnalisés)



Rappels Quarkus

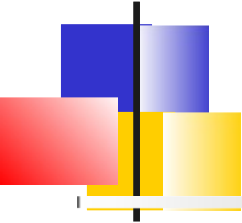
Positionnement
Quarkus vs SpringBoot
Développer avec Quarkus
Principales annotations
Configuration et profils



Introduction

Le modèle de programmation Quarkus est basé sur **CDI** : **Contexts and Dependency Injection** (Extension *Arc*)

- Les développeurs écrivent des beans dont le cycle de vie est géré par le framework (Pattern IoC)
- Des annotations permettent de déclarer, configurer et injecter les beans.
- D'autres annotations permettent d'y appliquer des aspects



Annotations CDI – Déclaration de Beans

@ApplicationScoped : Le plus courant. Une seule instance pour toute l'application, créée à la première utilisation (lazy). À utiliser par défaut pour services, repositories, clients externes.

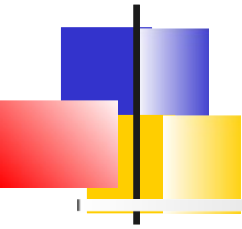
@Singleton : Similaire à @ApplicationScoped mais sans proxy client. Plus rapide à l'invocation, mais ne peut pas être remplacé à chaud par un mock en test. Bien pour les beans très sollicités sans besoin d'interception.

@Produces : Sur une méthode ou un champ, déclare que la valeur retournée est un bean injectable. Utilisé pour les beans techniques de configuration (similaire à @Bean en SpringBoot)

@RequestScoped : Une instance par requête HTTP (ou par contexte de requête). Utile pour porter des données liées à la requête courante.

@Dependent : Scope par défaut si rien n'est précisé. Une nouvelle instance à chaque injection. Le scope dépend du scope du bean qui l'injecte.

@SessionScoped et **@ConversationScoped** : Existent mais peu utilisés en Quarkus (orienté stateless/réactif).



Annotations CDI – Injection de dépendance

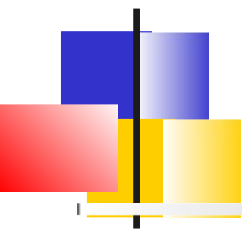
@Inject : L'annotation centrale, injecte un bean par type. Sur champ, constructeur ou setter.

@Named("nom") : Donne un nom au bean, utile pour le désambiguïser ou le référencer depuis des expressions.

@Qualifier : Pour créer ses propres qualifieurs quand plusieurs implémentations d'une même interface coexistent.

@Any et **Instance<T>** : Pour injecter dynamiquement toutes les implémentations d'un type, ou les résoudre programmatiquement.

A noter l'injection implicite par constructeur permet de se passer de
@Inject



Annotations CDI – Cycle de vie / Évènements

Cycle de vie :

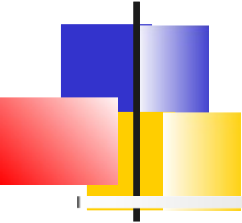
@PostConstruct : Méthode appelée après injection, pour initialiser le bean.

@PreDestroy : Méthode appelée avant destruction.

@Startup (spécifique Quarkus) : Force l'instanciation eager d'un bean *@ApplicationScoped* au démarrage de l'app, sans attendre la première injection. Pratique pour des initialisations.

Évènements :

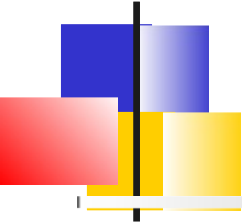
@Observes, @ObservesAsync : Sur un paramètre de méthode, fait de cette méthode un listener d'évènements CDI.



Annotations CDI – Configuration

@ConfigProperty (MicroProfile Config, intégré) : Injecte une propriété de configuration depuis *application.properties*, variables d'env, etc.

@ConfigMapping (spécifique Quarkus) : Mappe un groupe de propriétés vers une interface typée. Préférable à @ConfigProperty pour des configs structurées. Permet de faire de la validation de configuration

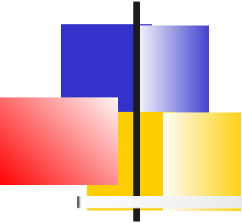


Annotations CDI – Intercepteurs

@Interceptor + @InterceptorBinding : Pour créer des intercepteurs réutilisables (logging, métriques, sécurité).

@Decorator : Pour les décorateurs

@Transactional (JTA) : Très utilisé, démarcation transactionnelle déclarative. Quarkus l'intercepte via CDI.

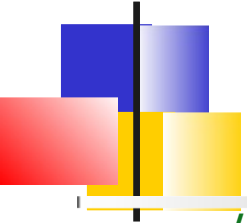


Annotations CDI – Conditions sur les beans

@IfBuildProperty, @UnlessBuildProperty : Conditionne la création de bean à la présence/absence d'une propriété de configuration. (Figé au build)

@IfBuildProfile, @UnlessBuildProfile : Conditionne la création de bean à l'activation/désactivation d'un profil. (Figé au build)

@LookupIfProperty, @LookupUnlessProperty : Bascule dynamique en fonction d'une propriété. (Résolu au runtime)



Un bean typique

// Annotation déclarant un bean et son cycle de vie

@ApplicationScoped

public class Translator {

// Injection de dépendance d'un autre bean

// Attention, le qualifier private n'est pas recommandé dans ce cas

@Inject

Dictionary dictionary;

// Intercepteur appliquant un cross-cutting concern

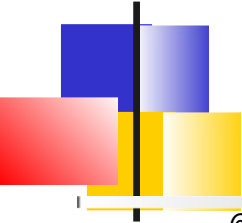
@Counted

String translate(String sentence) {

 // ...

}

}



Injection implicite

```
@ApplicationScoped
```

```
public class Translator {
```

```
    private final TranslatorHelper helper;
```

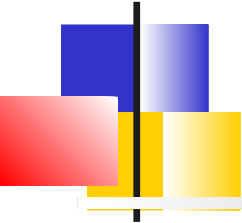
```
    // Injection par le constructeur
```

```
    Translator(TranslatorHelper helper) {
```

```
        this.helper = helper;
```

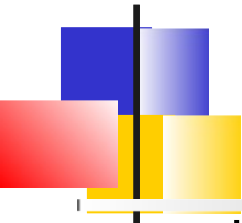
```
    }
```

```
}
```



Rappels Quarkus

Positionnement
Quarkus vs SpringBoot
Développer avec Quarkus
Principales annotations
Configuration et profils

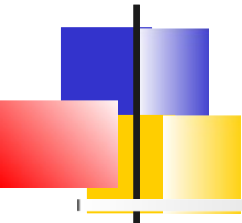


Sources de configuration

Les propriétés de configuration peuvent être définies à partir de plusieurs sources (par ordre décroissant) :

- **Propriétés système** : `-D` au démarrage
- **Variables d'environnement** : Passage en uppercase avec underscore
- **Fichier `.env`** dans le répertoire de travail actuel : Même règle que les variables d'environnement
- **Fichier de configuration « `ops` »** dans `$PWD/config/application.properties`
- **Fichier de configuration « `dev` »** `application.properties` dans le classpath
- **Fichier de configuration `MicroProfile Config`**
`META-INF/microprofile-config.properties` dans le classpath

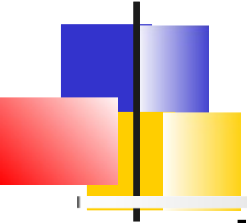
La configuration finale est l'agrégation des propriétés définies par toutes ces sources.



Sources additionnelles

Quarkus fournit des extensions supplémentaires qui couvrent d'autres formats de configuration :

- **YAML** (*quarkus-config-yaml*)
- **Consul** (*quarkus-config-consul*)
- **Spring Cloud Config** (*quarkus-spring-cloud-config-client*)
- **Kubernetes ConfigMap** et **Secrets** : (*quarkus-kubernetes-config*)
- **HashiCorp Vault** (*quarkus-vault*)

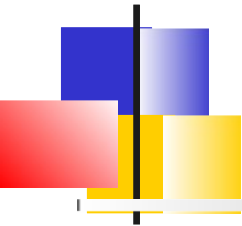


Expression pour les propriétés

Les valeurs de configuration peuvent utiliser des expressions spécifiées par la séquence **`${ ... }`**.

```
remote.host=quarkus.io
```

```
callable.url=https://${remote.host}/
```



Injection

@ConfigProperty permet de s'injecter une clé de configuration :

```
// Si clé pas présente, startup fails
```

```
@ConfigProperty(name = "greeting.message")
```

```
String message;
```

```
// Si clé pas présente, valeur par défaut
```

```
@ConfigProperty(name = "greeting.suffix", defaultValue="!")
```

```
String suffix;
```

```
// Si clé pas présente, Optional is Empty
```

```
@ConfigProperty(name = "greeting.name")
```

```
Optional<String> name;
```



Objets de configuration

Plusieurs propriétés peuvent être encapsulées dans une interface partageant le même préfixe. @ConfigMapping s'injecte ensuite comme un bean classique.

```
// Définition de server.host et  
server.port  
@ConfigMapping(prefix = "server")  
public interface Server {  
    String host();  
    int    port();  
}
```

```
// Injection comme un bean  
class BusinessBean {  
    @Inject  
    Server server;  
  
    public void businessMethod() {  
        String host = server.host();  
    }  
}
```



Structures avancées

Les interfaces se composent : imbrication, List/Set et Map sont supportés.

```
// Imbrication
@Configuration(prefix = "server")
public interface Server {
    String host();
    int    port();
    Log    log();

    interface Log {
        // server.log.enabled
        boolean enabled();
        String  suffix();
        boolean rotate();
    }

    // Map: server.form.<clé>
    Map<String, String> form();
}
```

```
// Collections (List / Set)
@Configuration(prefix = "server")
public interface ServerCollections {
    Set<Environment> environments();

    interface Environment {
        String name();
        List<App> apps();

        interface App {
            String name();
            List<String> services();
            Optional<List<String>> databases();
        }
    }
}
```



Valeurs par défaut & application.properties

@WithDefault fixe une valeur par défaut directement sur l'interface.

```
@ConfigMapping(prefix = "server")
```

```
public interface Server {
```

```
    @WithDefault("localhost")
```

```
    String host();
```

```
    @WithDefault("8080")
```

```
    int port();
```

```
    Map<String, String> form();
```

```
    Set<Environment> environments();
```

```
}
```

```
# application.properties
```

```
server.host=localhost
```

```
server.port=8080
```

```
# Map server.form.<key>
```

```
server.form.login-page=login.html
```

```
server.form.error-page=error.html
```

```
# Groupe imbriqué
```

```
server.log.enabled=true
```

```
server.log.suffix=.log
```

```
server.log.rotate=true
```

```
# Collections indexées
```

```
server.environments[0].name=dev
```

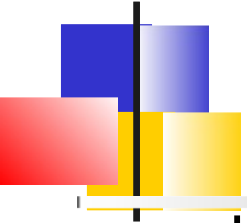
```
server.environments[0].apps[0].name=rest
```

```
server.environments[0].apps[0].services=bookstore,registration
```

```
server.environments[0].apps[0].databases=pg,h2
```

```
server.environments[0].apps[1].name=batch
```

```
server.environments[0].apps[1].services=stock,warehouse
```



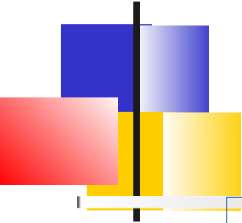
Validation

Une interface de configuration peut combiner des annotations de **Bean Validation** pour valider les valeurs de configuration

L'extension *quarkus-hibernate-validator* doit être présente

```
@ConfigMapping(prefix = "server")
interface Server {
    @Size(min = 2, max = 20)
    String host();

    @Max(10000)
    int port();
}
```

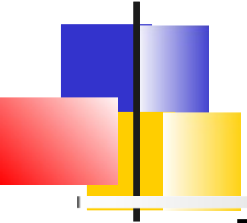



Les profils

Il est souvent nécessaire de configurer différemment en fonction de l'environnement cible. (intégration, production, etc..)

Les **profils de configuration** permettent plusieurs configurations dans le même fichier ou des fichiers séparés

Les profils sont ensuite activés via un nom au moment du build



Profil dans le nom de la propriété

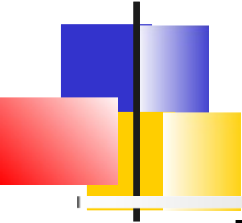
Pour pouvoir définir des propriétés avec le même nom, chaque propriété doit être précédée d'un **%**, le nom du profil et **.**

```
quarkus.http.port=9090
```

```
%dev.quarkus.http.port=8181
```

Les profils dans le fichier *.env* suivent la syntaxe suivante :

```
_ {PROFILE} _CONFIG_KEY=value
```



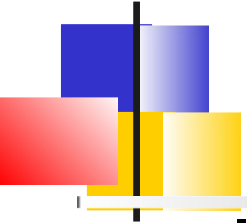
Profils par défaut

Par défaut, Quarkus propose trois profils, qui s'activent automatiquement dans certaines conditions :

- **dev** - Activé en mode développement (c'est-à-dire *quarkus dev*)
- **test** - Activé lors de l'exécution de tests
- **prod** - Le profil par défaut lorsqu'il n'est pas exécuté en mode développement ou test

On peut y ajouter ses profils personnalisés, il suffit de définir une propriété avec un nouveau nom de profil

%staging.quarkus.http.port=9999



Nommage de fichier

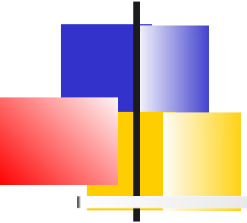
Il est également possible de regrouper toutes les configuration d'un profil dans un seul fichier

application-{profile}.properties

```
# application-staging.properties
```

```
quarkus.http.port=9190
```

```
quarkus.http.test-port=9191
```



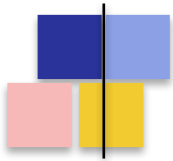
Activation du profil

Le profil d'exécution Quarkus est celui défini au moment du build

```
./mvnw package -Dquarkus.profile=staging
```

```
java -jar ./target/quarkus-app/quarkus-run.jar
```

= > Exécution avec le profil
staging



Programmation réactive

Problème de la concurrence

Le modèle réactif

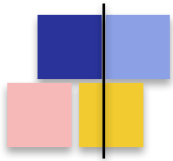
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

Quarkus et ReactiveX

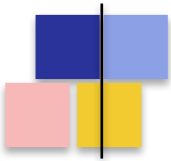
Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



Le modèle classique : 1 thread / requête

Servlet, Spring MVC, JAX-RS classique : **modèle synchrone bloquant**

- Chaque requête HTTP est traitée par un thread dédié, du début à la fin
- Pendant un appel I/O (BDD, REST, fichier), le thread reste assigné et bloqué
- Le thread **consomme sa pile mémoire (~1 Mo)** même lorsqu'il attend une réponse
- Capacité limitée par la taille du pool : Tomcat ~200 threads par défaut



Le mur de la concurrence

- Symptôme observé :

la latence explose au-delà d'un seuil alors que CPU et RAM ne sont pas saturés

- Cause :

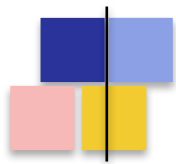
Tous les threads du pool sont bloqués **en attente d'I/O**

=> Les nouvelles requêtes s'empilent en file d'attente, puis timeout

=> Sous-utilisation matérielle typique : **CPU à 15 %, threads à 100 %**

- Augmenter le pool n'aide pas :

- context switch, contention mémoire, GC pressure
- Le coût d'un thread OS est élevé : *~1 Mo de stack, allocation noyau*



Inadapté aux architectures modernes

- Architectures microservices : **chaque service appelle d'autres services.**

Une requête entrante = N appels HTTP sortants en attente

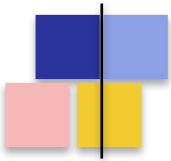
=> Multiplie d'autant le temps de blocage par requête

- Containers et serverless : **mémoire limitée, scaling horizontal**

Provisionner 200 threads x 1 Mo de stack = 200 Mo... juste pour attendre !

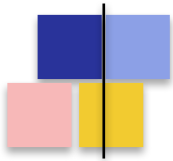
- Streaming et long-polling (SSE, WebSocket) : **des connexions ouvertes pendant des minutes voire des heures**

=> Impossible à tenir avec un thread par connexion



Deux réponses au problème

- Approche 1 — **Programmation réactive (non-bloquante)**
 - On ne bloque jamais un thread : on s'abonne au résultat futur
 - Quelques threads (event-loops) traitent des milliers de requêtes
 - Code asynchrone : Uni, Multi, callbacks, opérateurs
- Approche 2 — **Virtual Threads (Project Loom, Java 21)**
 - Threads ultra-légers gérés par la JVM, pas par l'OS
 - Code synchrone classique, mais sans coût de blocage
 - Compatible avec JDBC, Hibernate ORM, code existant
- ***Quarkus supporte les deux modèles*** — au choix selon le contexte



Programmation réactive

Problème de la concurrence

Le modèle réactif

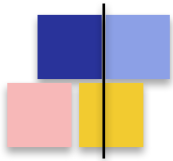
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

Quarkus et ReactiveX

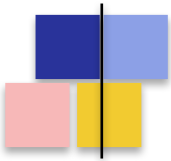
Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



Le Reactive Manifesto

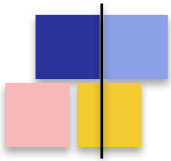
Publié en 2014, définit **4 propriétés** d'un système réactif :

- **Responsive** — répond rapidement et de manière prévisible
 - **Resilient** — reste réactif en cas de panne, isole les défaillances
 - **Elastic** — s'adapte à la charge, scale up et scale down
 - **Message-driven** — communication asynchrone par messages, faible couplage
- *En pratique : l'I/O non-bloquant et la composition d'événements asynchrones*



L'event loop : le cœur du modèle

- Inspiration : **Node.js, Netty, NGINX**
- Un petit nombre de threads ($\approx$ N coeurs CPU) gère TOUTES les requêtes
- Chaque opération I/O est **non-bloquante** : on déclenche et on rend la main
- Quand la donnée est disponible (event), un callback est rejoué sur l'event-loop
- Règle d'or : **JAMAIS de code bloquant sur un event-loop thread**
 - Sinon : tout le système se bloque (et pas seulement la requête courante)
- Quarkus s'appuie sur Eclipse Vert.x pour fournir ce modèle



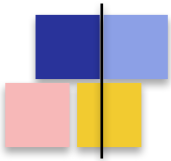
Bloquant vs Non-bloquant

Modèle bloquant

- 1 thread par requête
- ~200 threads max (Tomcat)
- Thread bloqué pendant l'I/O
- Coût mémoire : ~1 Mo/thread
- Code synchrone, simple à lire
- Debug facile (stack trace)
- Limité en concurrence

Modèle réactif

- Quelques event-loops ($\approx N$ coeurs)
- Des milliers de requêtes concurrentes
- Thread libéré pendant l'I/O
- Coût mémoire : très faible
- Code asynchrone (Uni, Multi)
- Debug : pile d'exécution éclatée
- Excellente scalabilité



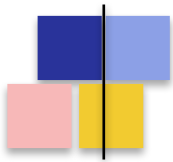
Avantages et limites

- **Avantages**

- Très haute concurrence avec peu de ressources
- Adapté au streaming, SSE, WebSocket, gateways
- Excellent fit pour les microservices qui appellent d'autres microservices
- Composition fine d'opérations asynchrones (parallel, retry, timeout...)

- **Limites**

- Courbe d'apprentissage : pensée asynchrone, opérateurs, lifecycle
- Stack traces peu lisibles, debug plus complexe
- Une seule librairie bloquante "oubliée" peut tuer la performance
- Pas toujours pertinent : pour du CRUD simple, le mode impératif suffit



Programmation réactive

Problème de la concurrence

Le modèle réactif

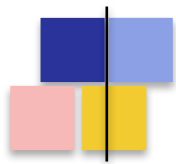
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

Quarkus et ReactiveX

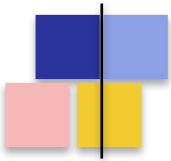
Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



Producteur rapide, consommateur lent

Cas typique en streaming : **le producteur émet plus vite que le consommateur ne traite**

- Exemple : un topic Kafka qui pousse 10 000 msg/s, un service qui en traite 1 000/s
- Exemple : une requête SQL qui ramène 1 M de lignes, un client plus lent
- **Sans régulation, conséquences inévitables :**
 - Empilement en file mémoire jusqu'à l'OutOfMemoryError
 - Latence qui grimpe (queueing delay)
 - Effet cascade : un service lent fait tomber tous les amonts
- *La back-pressure : **mécanisme pour que le consommateur dicte le rythme***



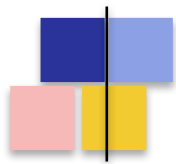
Le contrat de back-pressure

- Le consommateur appelle **request(n)** pour signaler qu'il peut absorber n éléments
- Le producteur **n'émet jamais plus que demandé**
- Quand le consommateur est prêt pour plus, il redemande
- Quand la source ne respecte pas la backpressure, on insère un opérateur de gestion du surplus côté consommateur

Stratégies usuelles côté Quarkus / Mutiny :

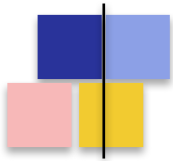
- BUFFER : on stocke temporairement (avec une borne)
- DROP : on jette les éléments en surplus
- LATEST : on ne garde que le dernier
- ERROR : on signale une erreur dès dépassement

Le bon choix dépend du métier (perte tolérée ? cohérence ? latence ?)



Pourquoi la back-pressure est centrale

- C'est ce qui **différencie un vrai système réactif d'un simple système asynchrone**
- L'asynchrone résout le problème du blocage CPU/thread
- La back-pressure résout le problème de la saturation mémoire
- Garantit la propriété « **resilient** » du Reactive Manifesto
- Permet le streaming sans risque pour le serveur
- Inscrit dans le contrat de la spec **Reactive Streams** (et donc de *java.util.concurrent.Flow*)



Programmation réactive

Problème de la concurrence

Le modèle réactif

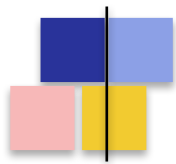
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

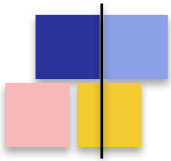
Quarkus et ReactiveX

Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



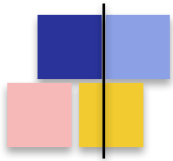
Reactive Streams : la spécification

- Initiative démarrée en 2013 par **Netflix, Lightbend, Pivotal, Red Hat**
- Objectif : **standardiser les flux asynchrones avec back-pressure non-bloquante**
- Spec minimaliste : 4 interfaces + TCK (Technology Compatibility Kit)
- **Implémentations majeures :**
 - Project Reactor (Spring) — Flux, Mono
 - RxJava 3 — Flowable, Observable
 - Akka Streams — Source, Flow, Sink
 - SmallRye Mutiny (Quarkus) — Uni, Multi
- **Intégrée à la JDK depuis Java 9** sous le package *java.util.concurrent.Flow*



Les 4 interfaces de l'API Flow

- **Publisher<T>** — source de données : accepte un Subscriber via *subscribe()*
- **Subscriber<T>** — consommateur : 4 callbacks
 - *onSubscribe(Subscription)* : appelé une fois à l'abonnement
 - *onNext(T)* : un nouvel élément est disponible
 - *onError(Throwable)* : terminal, erreur fatale
 - *onComplete()* : terminal, fin normale du flux
- **Subscription** — le canal de contrôle entre les deux
 - *request(long n)* : demande n éléments supplémentaires (back-pressure)
 - *cancel()* : interrompt l'abonnement
- **Processor<T,R>** — à la fois Publisher et Subscriber (étape intermédiaire)

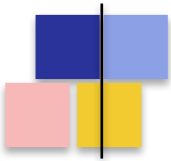


Cycle de vie d'un abonnement

1. *subscriber.onSubscribe(subscription)* — handshake initial
2. *subscriber.request(n)* — déclare sa capacité d'absorption
3. *subscriber.onNext(item)* — émet au plus *n* éléments
4. Subscriber peut rappeler *request(n)* à tout moment
5. Terminaison par *onComplete()*, *onError()* ou *cancel()*

Règle stricte : ***aucun callback ne doit bloquer le thread appelant***

C'est ce contrat qui rend les implémentations **interopérables**
(*Reactor* ↔ *RxJava* ↔ *Mutiny*)



Un Subscriber minimaliste

- Exemple d'un Subscriber qui consomme un flux d'entiers, élément par élément :

```
import java.util.concurrent.Flow.*;

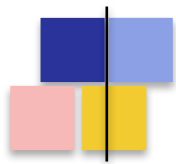
public class PrintSubscriber implements Subscriber<Integer> {

    private Subscription subscription;

    @Override
    public void onSubscribe(Subscription s) {
        this.subscription = s;
        s.request(1);                                // back-pressure : 1 a la fois
    }

    @Override
    public void onNext(Integer item) {
        System.out.println("Recu : " + item);
        subscription.request(1);                    // redemande apres traitement
    }

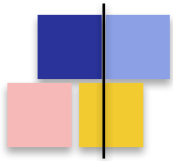
    @Override public void onError(Throwable t) { t.printStackTrace(); }
    @Override public void onComplete()          { System.out.println("Fin"); }
}
```

Pourquoi des API de plus haut niveau ?

- Flow / Reactive Streams = **le protocole bas-niveau**
- Suffisant pour interopérer, mais pas pour développer au quotidien
- Pas d'opérateurs (map, filter, retry, timeout, merge, combine...)
- **Les bibliothèques réactives ajoutent une couche fluide :**
 - Project Reactor : Flux<T>, Mono<T>
 - RxJava : Flowable<T>, Observable<T>
 - **SmallRye Mutiny** : **Uni<T>** (0 ou 1 élément) et **Multi<T>** (flux d'éléments) — c'est le choix de Quarkus

Toutes implémentent Reactive Streams : on peut convertir librement de l'une à l'autre



Programmation réactive

Problème de la concurrence

Le modèle réactif

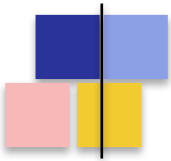
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

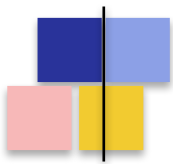
Quarkus et ReactiveX

Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



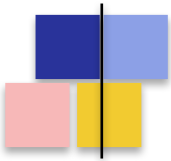
Project Loom : la promesse

- Initié en 2018, stabilisé en **Java 21 (LTS, septembre 2023)**
- Promesse : **la simplicité du code synchrone, la scalabilité du modèle réactif**
- **Un Virtual Thread est :**
 - un Thread Java classique au niveau API (`java.lang.Thread`)
 - mais géré par la JVM, pas par l'OS
 - ultra-léger : quelques centaines d'octets (vs ~1 Mo pour un thread OS)
 - on peut en créer des millions sans souci
- La JVM monte le virtual thread sur un **thread porteur (carrier thread)** quand il s'exécute, le démonte quand il bloque



Comment la JVM gère un blocage

- Quand le code virtuel appelle un I/O bloquant (*socket.read, JDBC, sleep...*) :
 - La JVM intercepte le blocage (la stdlib a été instrumentée)
 - Elle parke le virtual thread et libère le carrier thread
 - Le carrier thread est rendu au pool, libre d'exécuter un autre virtual thread
 - Quand l'I/O termine, le virtual thread est rendu schedulable
 - Il est repris sur n'importe quel carrier thread disponible
- Résultat : **le code paraît bloquant, mais le thread OS ne bloque jamais**
- Idéal pour : **JDBC, Hibernate ORM, appels HTTP synchrones**

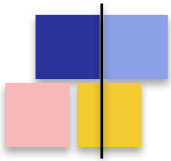


Code Virtual Threads en Java 21

```
// API equivalente a un Thread classique
Thread.startVirtualThread(() -> {
    String body = httpClient.send(request, BodyHandlers.ofString()).body();
    System.out.println(body);
});

// Executor dedie (utile pour gerer des milliers de taches)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i ->
        executor.submit(() -> {
            // chaque tache obtient son propre virtual thread
            return appelMicroservice(i);
        })
    );
}
```

Le code reste **synchrone et lisible**, mais la JVM peut soutenir des milliers de tâches concurrentes.



Réactif ou Virtual Threads ?

- **Virtual Threads — points forts**

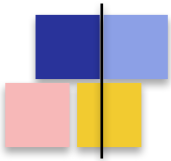
- Compatible avec tout l'écosystème Java bloquant (JDBC, Hibernate, librairies historiques)
- Code synchrone : lisible, débogage classique, stack traces complètes
- Courbe d'apprentissage quasi nulle pour les équipes

- **Réactif (Mutiny / Vert.x) — points forts**

- Back-pressure native, indispensable pour le streaming continu
- Composition fine (combinaisons, retry, timeout, parallélisme contrôlé)
- SSE, WebSocket, Kafka streaming : terrain de jeu naturel

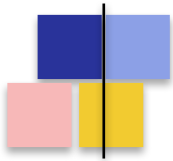
- *En pratique : ***pas un choix exclusif****

- Quarkus permet de mixer les 3 modèles : impératif, réactif, virtuel
- `@Blocking`, `@NonBlocking`, `@RunOnVirtualThread` (vu plus loin)



Synthèse du module

- Le modèle 1 thread / requête ne passe pas l'échelle **dans le cloud et les microservices**
- Le modèle réactif : **event-loop, I/O non-bloquant, programmation asynchrone par composition**
- La back-pressure : **le consommateur dicte le rythme, garantit la stabilité mémoire**
- Reactive Streams : **spec standard, intégrée à la JDK via `java.util.concurrent.Flow`**
- Virtual Threads : **alternative simple et scalable, complémentaire du réactif**



Programmation réactive

Problème de la concurrence

Le modèle réactif

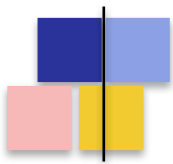
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

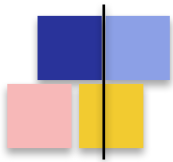
Quarkus et ReactiveX

Annotations `@Blocking`, `@NonBlocking`, `@RunOnVirtualThreads`



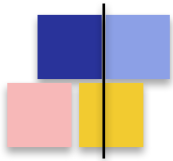
Eclipse Vert.x

- Toolkit **polyglotte (Java, Kotlin, JS, Groovy, Scala, Ruby)** pour construire des applications réactives sur la JVM
- Projet **Eclipse Foundation**, soutenu activement par Red Hat
- Inspiré de Node.js, mais multi-threadé : exploite tous les cœurs CPU
- Apporte : **event-loops, I/O non-bloquant, event bus, clients réactifs (HTTP, SQL, Kafka...)**
- *Dans Quarkus : **c'est le moteur d'exécution sous-jacent***
 - La couche HTTP, la base de données réactive, Kafka... tout passe par Vert.x
 - Souvent transparent pour le développeur : on écrit du Mutiny, Quarkus traduit



Vert.x au cœur de Quarkus

- Au démarrage, Quarkus instancie **une instance Vert.x** partagée par toute l'application
- Cette instance gère deux pools de threads distincts :
 - **Event-loop pool** — petite taille fixe (**2 × N cœurs CPU** par défaut)
 - Réservé au code non-bloquant : HTTP, réactif, callbacks Mutiny
 - JAMAIS bloquer un de ces threads, sinon dégradation globale
 - **Worker pool** — taille élastique (**20 threads par défaut, jusqu'à 200**)
 - Destiné au code bloquant : JDBC, fichiers, appels HTTP synchrones
 - Reçoit les tâches dispatchées par les event-loops quand nécessaire



Les deux pools de threads en image

Event-loop pool

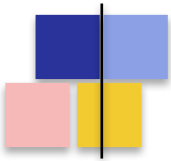
(I/O thread)

- Taille fixe : $2 \times N$ cœurs
- Threads nommés vert.x-eventloop-*
- Code 100 % non-bloquant
- Traite HTTP, callbacks réactifs
- Très haute concurrence
- Règle d'or : ne PAS bloquer

Worker pool

(executor thread)

- Taille élastique : $20 \rightarrow 200$
- Threads nommés executor-thread-*
- Code bloquant autorisé
- JDBC, fichiers, REST sync
- Concurrency limitée par la taille
- Bloquer ici est OK et attendu



Configuration des pools

- Tout se règle dans **application.properties**

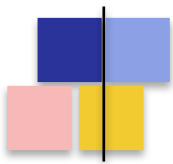
```
# Nombre d'event-loop threads (default : 2 x nb de coeurs)  
quarkus.vertx.event-loops-pool-size=8
```

```
# Worker pool (code bloquant)  
quarkus.vertx.worker-pool-size=20  
quarkus.thread-pool.max-threads=200  
quarkus.thread-pool.core-threads=8
```

```
# Detection automatique des blocages dans un event-loop  
# (warning au-dela du seuil en ms)  
quarkus.vertx.warning-exception-time=2S
```

```
# Activation des Virtual Threads (Java 21)  
quarkus.virtual-threads.enabled=true
```

Les valeurs par défaut conviennent à la majorité des cas — à n'ajuster qu'après mesure



Les 3 modèles d'exécution

- **1. Impératif classique (bloquant)**

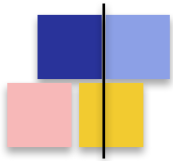
- La méthode retourne un type simple (String, Entity, List...)
- Exécutée sur un worker thread du pool exécuteur
- Code synchrone, JDBC, Hibernate ORM classique

- **2. Réactif (non-bloquant)**

- La méthode retourne Uni<T>, Multi<T>, CompletionStage<T>
- Exécutée directement sur un thread event-loop Vert.x
- Aucune ligne bloquante autorisée dans le corps de la méthode

- **3. Virtual Threads (Java 21)**

- Code bloquant classique, mais exécuté sur un virtual thread
- Active via `quarkus.virtual-threads.enabled=true`
- Activé par méthode avec `@RunOnVirtualThread`



Détection automatique du modèle

- Quarkus **devine le modèle d'exécution à partir du type de retour** de la méthode
- **Si la méthode retourne :**
 - Uni<T>, Multi<T>, CompletionStage<T>, Future<T> → event-loop thread
 - Type simple (String, MyEntity, List<T>...) → worker thread

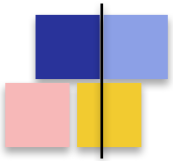
*Logique sous-jacente : **un type réactif suppose un code non-bloquant***

Cette détection peut être surchargée avec **@Blocking** ou **@NonBlocking**



Le piège : bloquer un event-loop

- Un thread event-loop bloqué = **toutes les requêtes qu'il pilote sont gelées**
- Comme il n'y a que $\sim 2 \times N$ cœurs event-loops, l'impact est massif
- **Quarkus surveille les event-loops :**
 - Trace un warning si une tâche dépasse `quarkus.vertx.warning-exception-time`
 - Affiche le thread bloqué et la stack au moment du blocage
- **Causes fréquentes :**
 - `Thread.sleep()`, `wait()`
 - Appel JDBC, RestClient synchrone, lecture fichier classique
 - Appel à `.await().indefinitely()` sur un Uni dans une méthode réactive
 - Bibliothèques tierces bloquantes (legacy, SDK propriétaires...)



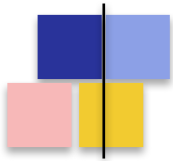
Le warning de blocage en action

- Exemple de log produit par Quarkus quand un event-loop est bloqué :

```
WARN [io.vertx.core.impl.BlockedThreadChecker]
Thread Thread[vert.x-eventloop-thread-3,5,main] has been blocked
for 2503 ms, time limit is 2000 ms
io.vertx.core.VertxException: Thread blocked
    at java.base@21/java.lang.Thread.sleep(Native Method)
    at fr.plb.demo.GreetingResource.hello(GreetingResource.java:18)
    at ...
```

Resolution :

- # - soit annoter la methode avec `@Blocking`
- # - soit déplacer l'I/O vers un Uni reactif
- # - soit utiliser `@RunOnVirtualThread`



Programmation réactive

Problème de la concurrence

Le modèle réactif

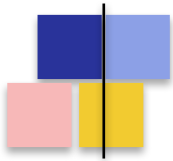
L'intérêt de la back-pressure

Reactive Streams et API Flow

Virtual Threads

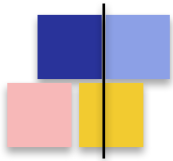
Quarkus et ReactiveX

**Annotations @Blocking, @NonBlocking,
@RunOnVirtualThreads**



Pourquoi @Blocking et @NonBlocking ?

- La détection automatique **fonctionne dans 80 % des cas**, mais pas toujours :
 - Une méthode renvoie un Uni mais contient un appel JDBC bloquant
 - Une méthode renvoie un type simple mais ne fait que des opérations en mémoire
 - On utilise un SDK historique qui ne propose pas d'API réactive
- **@Blocking** (package io.smallrye.common.annotation) : **force l'exécution sur le worker pool**
- **@NonBlocking** : **force l'exécution sur l'event-loop** (rare, mais utile dans Messaging notamment)
- Ces annotations s'appliquent sur *méthodes de Resources REST, listeners Messaging, gRPC, etc.*



@Blocking sur un endpoint REST

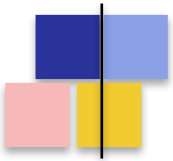
- Méthode qui retourne un Uni mais utilise une API JDBC bloquante :

```
import io.smallrye.common.annotation.Blocking;
import io.smallrye.mutiny.Uni;

@Path("/users")
public class UserResource {

    @Inject UserRepository repo;    // utilise JDBC classique (bloquant)

    @GET
    @Path("/{id}")
    @Blocking                        // execution forcee sur worker thread
    public Uni<User> getUser(@PathParam("id") Long id) {
        // Sans @Blocking, ce code tournerait sur un event-loop
        // et le repo.findById bloquerait toute l'application
        User user = repo.findById(id);
        return Uni.createFrom().item(user);
    }
}
```



@NonBlocking : forcer l'event-loop

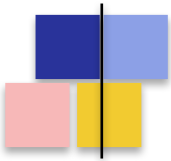
- Cas typique : **méthode qui retourne un type simple** mais qui ne fait que du calcul en mémoire
- Eviter le saut vers le worker pool améliore la latence et la concurrence

```
import io.smallrye.common.annotation.NonBlocking;

@Path("/echo")
public class EchoResource {

    @GET
    @Path("/{msg}")
    @NonBlocking // execution sur event-loop
    public String echo(@PathParam("msg") String msg) {
        return msg.toUpperCase(); // pas d'I/O, pas de risque
    }
}

// Sans @NonBlocking : execution sur worker thread (overhead inutile)
// Avec @NonBlocking : execution directement sur event-loop, latence reduite
```



Comment choisir le modèle ?

Réactif

Uni / Multi

Quand :

- Streaming, SSE
- Microservices très concurrents
- Composition d'appels

Atouts :

- Back-pressure
- Très haute concurrence

Limites :

- Courbe d'apprentissage
- Debug complexe

Impératif

@Blocking

Quand :

- CRUD classique
- Code legacy
- Charge modérée

Atouts :

- Code simple
- Debug facile
- Compatibilité totale

Limites :

- Concurrency limitée
- Coût mémoire/thread

Virtual Threads

@RunOnVirtualThread

Quand :

- JDBC, Hibernate
- Concurrency élevée sans réactif

Atouts :

- Code synchrone simple
- Forte concurrence
- Debug classique

Limites :

- Java 21+ requis
- Encore récent



@RunOnVirtualThread en pratique

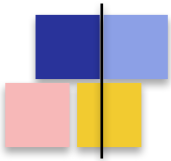
- Annotation **io.smallrye.common.annotation.RunOnVirtualThread**
- Nécessite Java 21 et `quarkus.virtual-threads.enabled=true`
- Excellent compromis pour conserver un code simple sans saturer le worker pool

```
import io.smallrye.common.annotation.RunOnVirtualThread;

@Path("/orders")
public class OrderResource {

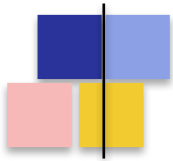
    @Inject OrderRepository repo;          // JDBC / Hibernate ORM classique

    @GET
    @Path("/{id}")
    @RunOnVirtualThread                    // execution sur virtual thread
    public Order getOrder(@PathParam("id") Long id) {
        // Code bloquant 100 % classique
        Order o = repo.findById(id);
        o.setItems(repo.loadItems(id));
        return o;
    }
}
```



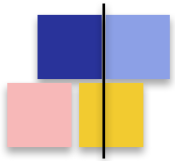
Pièges et bonnes pratiques

- **@Blocking sur une méthode qui retourne un Uni :**
 - Parfaitement valide, c'est même le cas d'usage principal
 - La souscription au Uni se fait depuis un worker thread
- **Ne pas mélanger les modèles dans une même méthode**
 - `.await().indefinitely()` sur un Uni dans du code event-loop = blocage garanti
 - Préférer composer avec `map()`, `flatMap()`, `onItem().transformToUni()`
- **Cohérence de toute la chaîne :**
 - Si un endpoint est `@NonBlocking`, ses services et repositories doivent l'être aussi
 - Sinon, le warning `BlockedThreadChecker` finira par sonner en production
- *Le profiler reste votre meilleur ami : **mesurer avant d'arbitrer***



Synthèse du module

- Quarkus s'appuie sur **Eclipse Vert.x** comme moteur d'exécution réactif
- Deux pools de threads : **event-loops (non-bloquant)** et **worker pool (bloquant)**
- Le modèle d'exécution est **déTECTÉ automatiquement** à partir du type de retour
- **@Blocking** et **@NonBlocking** permettent de surcharger ce comportement au cas par cas
- **@RunOnVirtualThread** offre une 3ème voie : **code simple, concurrence élevée**



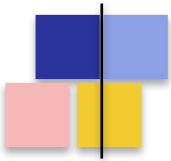
Mutiny

Concepts de base

Composition et gestion d'erreurs

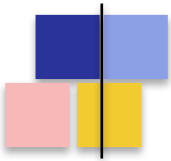
Concepts avancés

Tests



Qu'est-ce que Mutiny ?

- Mutiny est la **bibliothèque réactive intégrée à Quarkus**, développée par SmallRye (Red Hat)
- Implémente **Reactive Streams** — interopérable avec Reactor, RxJava, l'API Flow
- **Philosophie** : une API **guidée par les événements** plutôt que par des centaines d'opérateurs
 - Lecture naturelle : `onItem()`, `onFailure()`, `onCompletion()`...
 - Auto-complétion IDE explicite : on découvre l'API par tabulation
 - Stack traces enrichies en mode debug
- **Deux types fondamentaux** : *Uni<T>* et *Multi<T>*



Uni<T> et Multi<T>

Uni<T>

0 ou 1 item, puis fin

Représente :

- Un résultat unique futur
- Une opération qui se termine

Cas d'usage :

- Réponse d'un appel REST
- Résultat d'une requête SQL
- Lecture d'un document
- Action sans valeur (Uni<Void>)

Équivalents :

- Mono (Reactor)
- Single (RxJava)
- CompletionStage

Multi<T>

0 à N items, puis fin ou erreur

Représente :

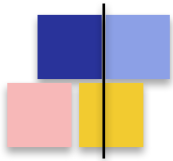
- Un flux d'éléments dans le temps
- Implémente Publisher (Flow)

Cas d'usage :

- Stream de lignes SQL
- Server-Sent Events (SSE)
- Messages Kafka
- WebSocket entrant

Équivalents :

- Flux (Reactor)
- Flowable (RxJava)
- Stream (mais non-bloquant)



Créer un Uni

- Le point d'entrée est **Uni.createFrom()**

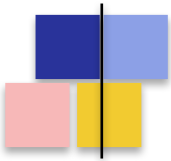
```
// A partir d'une valeur immediate
Uni<String> u1 = Uni.createFrom().item("hello");

// A partir d'un Supplieur (lazy : evalue a la souscription)
Uni<String> u2 = Uni.createFrom().item(() -> appelCouteux());

// A partir d'un echec
Uni<Integer> u3 = Uni.createFrom().failure(new IOException("boom"));

// Valeur nulle / Void
Uni<Void> u4 = Uni.createFrom().nullItem();
Uni<Void> u5 = Uni.createFrom().voidItem();

// Depuis du code existant
Uni<String> u6 = Uni.createFrom().completionStage(stage);
Uni<String> u7 = Uni.createFrom().future(future);
Uni<Person> u8 = Uni.createFrom().publisher(publisher);
```



Créer un Multi

- Le point d'entrée est **Multi.createFrom()**

// A partir d'items connus

```
Multi<Integer> m1 = Multi.createFrom().items(1, 2, 3, 4, 5);
```

// Depuis un Iterable / Stream

```
Multi<Integer> m2 = Multi.createFrom().iterable(List.of(1, 2, 3));
```

```
Multi<Integer> m3 = Multi.createFrom().items(myStream);
```

// Plage de valeurs

```
Multi<Integer> m4 = Multi.createFrom().range(0, 100);
```

// Flux periodique

```
Multi<Long> ticks = Multi.createFrom()  
    .ticks().every(Duration.ofSeconds(1));
```

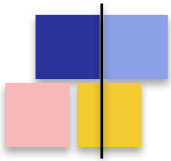
// Echec / Vide

```
Multi<String> m5 = Multi.createFrom().failure(new RuntimeException("ko"));
```

```
Multi<String> m6 = Multi.createFrom().empty();
```

// Generation (avec etat partage)

```
Multi<Integer> m7 = Multi.createFrom()  
    .generator(() -> 0, (s, e) -> { e.emit(s); return s + 1; });
```



Un pipeline déclaratif

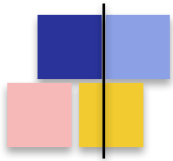
- Mutiny **ne déclenche rien tant qu'on ne s'abonne pas**
- On construit une suite d'opérations — c'est une description, pas une exécution

```
// Construction du pipeline (rien ne se passe ici)
Uni<String> pipeline = Uni.createFrom().item("hello")
    .onItem().transform(s -> s.toUpperCase())
    .onItem().transform(s -> "Message: " + s);

// A ce stade : aucune execution, aucun thread consomme, aucune I/O

// Declenchement avec subscribe
pipeline.subscribe().with(
    value    -> System.out.println("Resultat = " + value),
    failure  -> failure.printStackTrace()
);
```

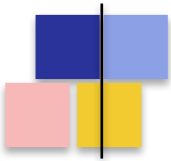
Dans un endpoint REST Quarkus, **on ne fait jamais .subscribe() soi-même** — c'est Quarkus qui le fait quand il produit la réponse HTTP.



Les groupes d'événements Mutiny

- Mutiny structure son API autour des **événements qui peuvent survenir**
 - **onItem()** — un nouvel élément est disponible
 - **onFailure()** — une erreur s'est produite
 - **onFailure(Class<?>)** — erreur d'un type spécifique
 - **onCompletion()** — fin normale du flux (Multi)
 - **onSubscription()** — abonnement initial
 - **onCancellation()** — le consommateur s'est désabonné
 - **onTermination()** — fin (normale, erreur ou annulation)

*Chaque groupe expose des opérations adaptées : **transform, invoke, call, recover, retry...***



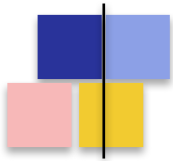
Les opérateurs essentiels

- **Sur onItem() :**

- **transform(fn)** — alias map(fn), transformation synchrone item → item
- **transformToUni(fn)** — alias flatMap(fn), chaîne un appel asynchrone
- **transformToMulti(fn)** — expose un item en un flux
- **invoke(fn)** — effet latéral synchrone, ne modifie pas le flux
- **call(fn)** — effet latéral asynchrone, attend la fin du Uni retourné

- **Spécifique à Multi :**

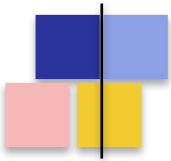
- **filter(predicate)** — ne garde que les items qui matchent
- **select().first(n) / select().last(n)** — n premiers / derniers
- **skip(n)** — ignore les n premiers
- **collect().asList() / asMap()** — agrégation en Uni<Collection>
- **group().intoLists().of(n)** — regroupement par paquets



Les opérateurs essentiels — en pratique

```
// === Uni : enchaîner des traitements et réagir aux erreurs ===
Uni<UserDto> user = userRepo.findById(id)           // Uni<User>
    .onItem().transform(User::toDto)                // map : item -> item (sync)
    .onItem().transformToUni(this::enrich)           // flatMap : chaîne un appel async
    .onItem().invoke(this::onLoaded)                 // effet de bord SYNCHROME
    .onItem().call(auditService::record)             // effet de bord ASYNCHROME
    .onFailure(NotFoundException.class)              // ciblage par type d'exception
    .recoverWithItem(UserDto.empty());              // valeur de repli sur erreur

// === Multi : filtrer, limiter, agréger en un Uni ===
Uni<List<String>> noms = userRepo.streamAll()         // Multi<User>
    .select().where(User::isActive)                 // filter : garde les actifs
    .select().first(20)                             // n premiers items du flux
    .onItem().transform(User::name)                  // map sur chaque item
    .collect().asList();                             // agrège le flux -> Uni<List>
```



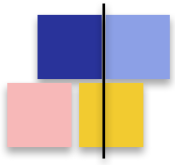
Exemple complet — Uni

```
@Path("/users")
public class UserResource {

    @Inject UserService userService;    // retourne des Uni

    @GET
    @Path("/{id}")
    public Uni<UserDto> getUser(@PathParam("id") Long id) {
        return userService.findById(id)
            .onItem().transform(this::toDto)
            .onItem().invoke(u -> log.infof("User found: %s", u.getName()))
            .onFailure(NotFoundException.class)
                .recoverWithItem(UserDto.empty())
            .onFailure().invoke(t -> log.errorf("Erreur : %s", t.getMessage()));
    }

    private UserDto toDto(User user) { /* ... */ }
}
```



Mutiny

Concepts de base

Composition et gestion d'erreurs

Concepts avancés

Tests



Composer des opérations asynchrones

Le problème

Une requête métier déclenche rarement un seul appel. Elle enchaîne plusieurs étapes asynchrones : récupérer un utilisateur, puis ses commandes, puis calculer un score...

Deux modes de composition

Séquentiel

```
chain() / transformToUni()
```

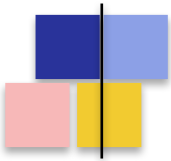
Le second appel a besoin du résultat du premier. Les étapes s'exécutent l'une après l'autre.

Parallèle

```
Uni.combine().all().unis(...)
```

Les appels sont indépendants. Ils se lancent simultanément, le résultat global est disponible quand tous ont terminé.

Bien choisir entre les deux est la clé de la performance : un chaînage séquentiel inutile multiplie la latence.

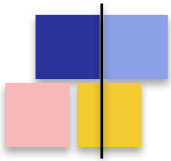


Chaînage séquentiel

- **transformToUni() / flatMap()** enchaîne deux appels asynchrones
- Le second démarre uniquement quand le premier a réussi

```
// Recuperer l'utilisateur, puis ses commandes, puis enrichir
Uni<EnrichedUser> result = userService.findById(id)
    .onItem().transformToUni(user ->
        orderService.findByUser(user.getId())
            .onItem().transform(orders -> new EnrichedUser(user, orders))
    )
    .onItem().transformToUni(eu ->
        scoringService.computeScore(eu)
            .onItem().transform(score -> eu.withScore(score))
    );
```

```
// Equivalent en style 'chain' (Mutiny >= 1.5)
Uni<EnrichedUser> result2 = userService.findById(id)
    .chain(user -> orderService.findByUser(user.getId())
        .map(orders -> new EnrichedUser(user, orders)))
    .chain(eu -> scoringService.computeScore(eu)
        .map(score -> eu.withScore(score)));
```



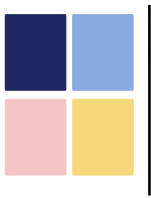
Combinaison parallèle

- **Uni.combine().all().unis(...)** lance plusieurs Uni **en parallèle**
- Le résultat est disponible quand TOUS les Uni ont émis leur item

```
// Trois appels en parallele, agreges en un Profile
Uni<Profile> profile = Uni.combine().all().unis(
    userService.findById(id),
    ordersService.recentOrders(id),
    prefService.getPreferences(id)
)
    .with((user, orders, prefs) -> new Profile(user, orders, prefs));

// Variante : le premier qui repond gagne
Uni<String> fastest = Uni.combine().any().of(
    callPrimaryEndpoint(),
    callSecondaryEndpoint(),
    callTertiaryEndpoint()
);

// Sur un Multi : combiner deux flux en un seul
Multi<Tuple2<A,B>> zipped = Multi.createBy().combining()
    .streams(streamA, streamB).asTuple();
```



Gérer les défaillances d'un flux

Le problème

Dans un système distribué, une étape peut échouer : timeout réseau, service indisponible, donnée absente. Sans stratégie, l'erreur d'un seul maillon casse toute la chaîne.

La réponse Mutiny

Le groupe `onFailure()` expose toutes les stratégies de gestion d'erreur, séparées du chemin nominal `onItem()`.

Récupérer

`recoverWithItem` / `recoverWithUni`

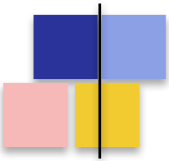
Substituer l'erreur par une valeur par défaut, un cache, ou un service de secours.

Résister

`retry` / `ifNoItem().after` / `timeout`

Réessayer avec back-off, limiter le temps d'attente, basculer après échec : les patterns de résilience classiques.

La séparation `onItem` / `onFailure` rend visible ce qui était noyé dans un `try/catch` impératif.



Gestion d'erreurs : récupération

- **onFailure()** donne accès à toutes les stratégies de récupération

```
// Substitution par une valeur par défaut
uni.onFailure().recoverWithItem(User.guest());

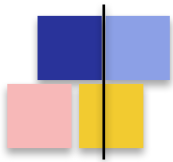
// Substitution avec acces a l'exception
uni.onFailure().recoverWithItem(err -> User.empty(err.getMessage()));

// Substitution par un autre Uni (fallback async)
uni.onFailure().recoverWithUni(err -> cacheService.getLastKnown(id));

// Recuperation conditionnelle par type d'exception
uni.onFailure(TimeoutException.class).recoverWithItem(User.empty())
  .onFailure(NotFoundException.class).recoverWithItem(User.guest())
  .onFailure().recoverWithUni(this::callFallbackService);

// Transformer l'exception (sans recuperer)
uni.onFailure().transform(BusinessException::new);

// Log + propagation
uni.onFailure().invoke(err -> log.errorf(err, "echec"));
```

Retry, timeout, fallback

- Mutiny intègre nativement les **patterns de résilience** classiques

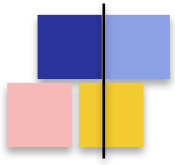
```
// Retry simple : 3 tentatives
uni.onFailure().retry().atMost(3);

// Retry avec backoff exponentiel + jitter
uni.onFailure().retry()
    .withBackOff(Duration.ofMillis(200), Duration.ofSeconds(5))
    .withJitter(0.5)
    .atMost(5);

// Retry illimite avec deadline
uni.onFailure().retry()
    .withBackOff(Duration.ofMillis(100))
    .expireIn(Duration.ofSeconds(30).toMillis());

// Timeout : echoue si pas de reponse en N secondes
uni.ifNoItem().after(Duration.ofSeconds(3)).fail();

// Timeout + fallback automatique
uni.ifNoItem().after(Duration.ofSeconds(3))
    .recoverWithItem(User.empty());
```



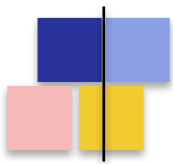
Mutiny

Concepts de base

Composition et gestion d'erreurs

Concepts avancés

Tests



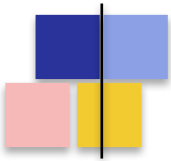
invoke() vs call() — l'effet latéral

- Les deux servent à exécuter un **effet de bord sans modifier le flux** (log, métrique, audit...)
- **invoke(Consumer)** — **synchrone**, retourne *void*, l'item est propagé après exécution
- **call(Function -> Uni)** — **asynchrone**, l'item est propagé après que le Uni retourné émet

```
// invoke : log synchrone, n'attend rien
multi.onItem().invoke(item -> log.infof("Recu : %s", item))

// call : effet asynchrone, ex. ecriture en BDD avant de propager
multi.onItem().call(item ->
    auditService.recordAsync(item)    // retourne Uni<Void>
)
// l'item n'est propagé qu'après que l'audit est écrit

// Pour un effet sans valeur de retour (Void)
uni.invoke(() -> metricRegistry.counter("calls").inc())
```



Convertir entre Uni et Multi

- Très fréquent dans les chaînes de traitement (*REST* → *BDD* → *stream*)

```
// Multi -> Uni (collecter en liste)
Uni<List<User>> users = userMulti.collect().asList();

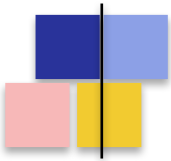
// Multi -> Uni (premier element)
Uni<User> first = userMulti.toUni();

// Multi -> Uni (agreger)
Uni<Long> count = userMulti.collect().with(Collectors.counting());

// Uni -> Multi (devient un flux d'un seul element)
Multi<User> single = userUni.toMulti();

// Uni<List<T>> -> Multi<T> (eclater une liste en flux)
Multi<User> stream = userListUni
    .onItem().transformToMulti(list ->
        Multi.createFrom().iterable(list)
    );

// Multi de Multi -> Multi (aplatir un flux de flux)
Multi<Order> all = userMulti
    .onItem().transformToMultiAndConcatenate(u -> ordersFor(u));
```



await() — sortir du monde réactif

- Mutiny permet de **bloquer volontairement** un Uni pour récupérer son résultat
- À utiliser **uniquement sur worker thread** — **JAMAIS sur un event-loop**

```
// Bloque indefiniment jusqu'a obtenir l'item ou l'echec
String value = uni.await().indefinitely();

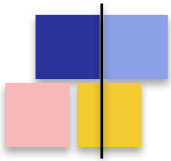
// Bloque avec timeout (TimeoutException si depasse)
String value2 = uni.await().atMost(Duration.ofSeconds(5));

// Recupere un Optional (null devient Optional.empty)
Optional<String> opt = uni.await().asOptional().indefinitely();

// Pour un Multi : collecter de maniere bloquante
List<Integer> all = multi.collect().asList()
    .await().atMost(Duration.ofSeconds(10));

// Stack trace explicite si on bloque un event-loop :
//   IllegalStateException: The current thread cannot be blocked
```

Cas typiques : tests, scripts CLI, code legacy qu'on adapte progressivement



Contrôler le threading

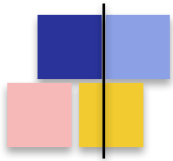
Deux opérateurs pour décider **sur quel thread** un Uni / Multi s'exécute :

- **emitOn(executor)** — bascule l'émission des items sur un autre executor
- **runSubscriptionOn(executor)** — exécute la souscription (et donc la création) sur un autre executor

```
// Decharger un calcul lourd vers le worker pool
Uni<Report> reactive = repo.fetchData()
    .emitOn(Infrastructure.getDefaultWorkerPool())
    .onItem().transform(this::heavyCpuComputation)
    .emitOn(Infrastructure.getDefaultExecutor());

// Forcer la creation sur un thread bloquant (legacy)
Uni<String> u = Uni.createFrom().item(() -> blockingCall())
    .runSubscriptionOn(Infrastructure.getDefaultWorkerPool());

// En pratique, dans Quarkus : preferer @Blocking ou @RunOnVirtualThread
// emitOn / runSubscriptionOn restent utiles pour des cas pointus
```



Memoize et multi-souscription

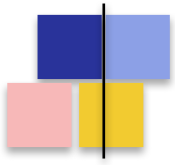
- Par défaut, chaque souscription **rejoue toute la pipeline**
- Si l'opération coûte cher, on peut mémoriser le résultat

```
// Sans memoize : appel rejoue a chaque souscription
Uni<Token> uni = authService.fetchToken(); // 3 souscriptions = 3 appels

// Avec memoize : un seul appel, resultat cache pour les suivants
Uni<Token> cached = authService.fetchToken()
    .memoize().indefinitely();

// Memoize avec expiration
Uni<Token> withTtl = authService.fetchToken()
    .memoize().atLeast(Duration.ofMinutes(5));

// Multi : transformer en HotStream (un seul producteur, plusieurs consommateurs)
Multi<Event> hot = sourceMulti.broadcast().toAllSubscribers();
```



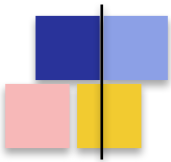
Mutiny

Concepts de base

Composition et gestion d'erreurs

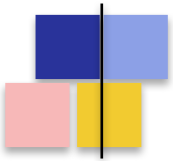
Concepts avancés

Tests



Tester du code réactif

- **Trois approches selon le contexte :**
- **1. await() dans le test** — le plus simple, bloque dans le thread JUnit
 - Acceptable car les tests s'exécutent hors d'un event-loop
 - Lecture du test très proche d'un test classique
- **2. UniAssertSubscriber / AssertSubscriber** — souscription contrôlée, assertions fines
 - Permet de tester les événements (souscription, items, fin, erreur)
 - Idéal pour vérifier la back-pressure d'un Multi
- **3. RestAssured + endpoints Quarkus** — tests d'intégration HTTP classiques
 - Quarkus gère lui-même la souscription, on teste la réponse HTTP



Tester avec await()

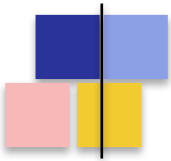
```
@QuarkusTest
class UserServiceTest {

    @Inject UserService userService;

    @Test
    void findById_returnsUser() {
        User user = userService.findById(1L)
            .await().atMost(Duration.ofSeconds(5));

        assertThat(user.getName()).isEqualTo("Alice");
    }

    @Test
    void findById_unknown_throwsNotFound() {
        assertThatThrownBy(() ->
            userService.findById(999L).await().indefinitely()
        ).isInstanceOf(NotFoundException.class);
    }
}
```



UniAssertSubscriber

- Pour valider le comportement événementiel d'un Uni sans bloquer :

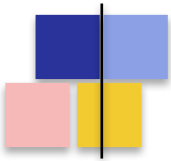
```
import io.smallrye.mutiny.helpers.test.UniAssertSubscriber;

@Test
void findById_emitsItem() {
    UniAssertSubscriber<User> sub = userService.findById(1L)
        .subscribe().withSubscriber(UniAssertSubscriber.create());

    sub.awaitItem(Duration.ofSeconds(2))
        .assertCompleted()
        .assertItem(new User(1L, "Alice"));
}

@Test
void findById_emitsFailure_whenMissing() {
    UniAssertSubscriber<User> sub = userService.findById(999L)
        .subscribe().withSubscriber(UniAssertSubscriber.create());

    sub.awaitFailure()
        .assertFailedWith(NotFoundException.class, "User 999 not found");
}
```



AssertSubscriber pour Multi

- Indispensable pour tester un **Multi avec back-pressure contrôlée**

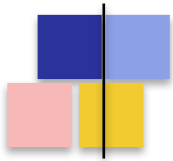
```
import io.smallrye.mutiny.helpers.test.AssertSubscriber;

@Test
void streamOrders_emitsAllItems() {
    AssertSubscriber<Order> sub = orderService.streamOrders()
        .subscribe().withSubscriber(AssertSubscriber.create(10));
    //                                     ^^
    //                               request(10) initial : back-pressure

    sub.awaitCompletion()
        .assertCompleted()
        .assertItems(order1, order2, order3);
}

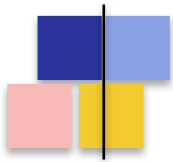
@Test
void streamOrders_respectsBackpressure() {
    AssertSubscriber<Order> sub = orderService.streamOrders()
        .subscribe().withSubscriber(AssertSubscriber.create(2));

    sub.awaitNextItems(2);
    sub.assertNotTerminated();           // pas de fin tant qu'on demande pas
    sub.request(Long.MAX_VALUE);
    sub.awaitCompletion();
}
```



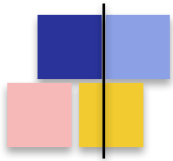
UniAsserter — tester sur contexte Vert.x

```
// UniAssertSubscriber -> Mutiny pur (await/getItem). Ici : UniAsserter,  
// helper Quarkus pour tester sur le contexte Vert.x .  
  
@Test  
@RunOnVertxContext // sinon asserter == null  
void testPersist(UniAsserter asserter) { // injecté en paramètre  
    asserter.assertThat(  
        () -> Panache.withTransaction( // Uni<Fruit> en transaction  
            () -> new Fruit("pomme").persist()),  
        fruit -> assertNotNull(fruit.id)); // assertion sur le résultat  
  
    asserter.assertFailedWith(  
        () -> Fruit.findById(-1L), // Uni dont on attend l'échec  
        NoResultException.class); // type d'exception attendu  
}
```



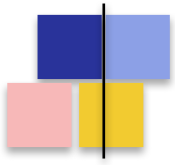
Bonnes pratiques de test

- Toujours mettre un timeout sur **await()**
 - `await().atMost(...)` plutôt que `await().indefinitely()` en CI
 - Évite de faire traîner un test bloqué en cas de bug
- Tester aussi les **cas d'échec et de timeout**
 - `retry`, `recover`, `fallback` : les vérifier explicitement
 - Utiliser Mockito ou WireMock pour simuler les latences et erreurs
- **Pour Multi** : vérifier le nombre d'items, l'ordre, la terminaison
 - `AssertSubscriber.create(n)` : tester la back-pressure réelle
- **Endpoints REST** : `RestAssured` + `@QuarkusTest` — pas besoin de toucher Mutiny



Synthèse du module

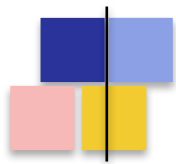
- Mutiny structure son API autour des **événements (onItem, onFailure...)**, pas des opérateurs
- **Uni<T>** pour 0 ou 1 item, **Multi<T>** pour les flux 0..N items
- Pipeline déclaratif : **rien ne s'exécute tant qu'on ne s'abonne pas**
- Composition : **chain pour le séquentiel, combine pour le parallèle**
- Résilience native : **retry, timeout, recover, fallback**
- Tests : **await() en simple, UniAssertSubscriber / AssertSubscriber en avancé**
- Suite : ***mettre Mutiny en action dans Quarkus REST, persistance et messaging***



Persistence

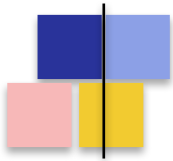
Hibernate et Panache

NoSQL



Hibernate Reactive : vue d'ensemble

- **Variante 100 % non-bloquante d'Hibernate ORM**, construite sur les **clients SQL réactifs Vert.x**
- Drivers réactifs : PostgreSQL, MySQL, MariaDB, MS SQL Server, DB2, Oracle
- **Différences notables avec Hibernate ORM classique :**
 - API basée sur Uni<T> et Multi<T> — pas de blocage
 - Pas de JTA, donc pas de @Transactional standard — voir section dédiée
 - Pas de chargement lazy implicite : les relations sont explicitement fetched
 - Mutiny.SessionFactory injecté à la place de EntityManagerFactory
- **Extensions à activer :**
 - quarkus-hibernate-reactive (le moteur ORM)
 - quarkus-reactive-pg-client / -mysql-client / -mssql-client... (le driver)



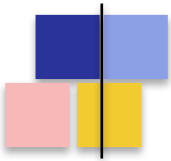
Configuration et entité

```
# application.properties
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=quarkus_test
quarkus.datasource.password=quarkus_test
quarkus.datasource.reactive.url=postgresql://localhost:5432/mydb
quarkus.datasource.reactive.max-size=20
quarkus.hibernate-orm.database.generation=drop-and-create

// Entite JPA classique (memes annotations qu'en mode bloquant)
@Entity
@Table(name = "persons")
public class Person {
    @Id @GeneratedValue
    public Long id;

    @Column(nullable = false)
    public String name;

    public LocalDate birthDate;
}
```



Usage de Mutiny.SessionFactory

```
import org.hibernate.reactive.mutiny.Mutiny;

@ApplicationScoped
public class PersonRepository {

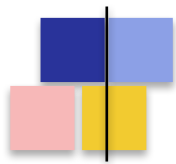
    @Inject Mutiny.SessionFactory sf;

    public Uni<List<Person>> findAll() {
        return sf.withSession(session ->
            session.createQuery("from Person", Person.class).getResultList()
        );
    }

    public Uni<Person> findById(Long id) {
        return sf.withSession(session -> session.find(Person.class, id));
    }

    public Uni<Person> create(Person p) {
        return sf.withTransaction((session, tx) ->
            session.persist(p).replaceWith(p)
        );
    }
}

// Note : withSession() ouvre la session, withTransaction() ouvre session + tx
```



Panache Reactif : moins de boilerplate

Panache simplifie **l'usage d'Hibernate Reactive** en proposant deux styles :

- **1. Active Record (PanacheEntity)**

- L'entité hérite de PanacheEntity et expose elle-même persist(), delete()...
- Méthodes statiques : findById(), findAll(), find("name", x), count()...
- Code très concis, idéal pour des CRUD simples

- **2. Repository (PanacheRepository)**

- Sépare l'entité (POJO JPA) et le repository qui implémente PanacheRepository<E>
- Mêmes méthodes raccourcies, mais accessibles depuis le repository injecté
- Préféré pour les architectures hexagonales / DDD

Extension : **quarkus-hibernate-reactive-panache** (différente de la version bloquante !)

- *Toutes les méthodes retournent **Uni<T>** ou **Multi<T>***



Panache Active Record Pattern

```
import io.quarkus.hibernate.reactive.panache.PanacheEntity;

@Entity
public class Person extends PanacheEntity {    // id Long auto-generate

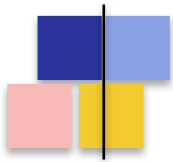
    public String name;
    public LocalDate birthDate;

    // Methodes statiques metier
    public static Uni<Person> findByName(String name) {
        return find("name", name).firstResult();
    }

    public static Uni<List<Person>> findAlive() {
        return list("status", Status.ALIVE);
    }

    public static Uni<Long> deleteStefs() {
        return delete("name", "Stef");
    }
}

// Utilisation : Person.findById(1L), Person.listAll(), person.persist()...
```



Panache Repository

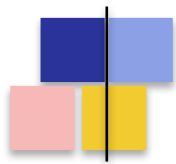
```
// L'entite reste un POJO JPA classique
@Entity
public class Person {
    @Id @GeneratedValue
    public Long id;
    public String name;
    public LocalDate birthDate;
}

// Le repository implemente PanacheRepository<E>
@ApplicationScoped
public class PersonRepository implements PanacheRepository<Person> {

    public Uni<Person> findByName(String name) {
        return find("name", name).firstResult();
    }

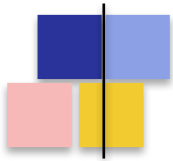
    public Uni<List<Person>> findRecent(LocalDate since) {
        return list("birthDate >= ?1 order by birthDate desc", since);
    }
}

// Utilisation : @Inject PersonRepository repo; repo.findById(1L);
```



Transactions réactives : le contexte

- **@Transactional (JTA)** n'est pas supportée par Hibernate Reactive
 - JTA s'appuie sur des ThreadLocal — incompatible avec le modèle event-loop
 - Une requête réactive peut migrer de thread en cours d'exécution
- **Trois alternatives :**
 - **@WithTransaction** sur une méthode qui retourne un Uni — la plus simple
 - **Panache.withTransaction(() -> ...)** — version programmatique
 - **sf.withTransaction(...)** sur Mutiny.SessionFactory — sans Panache
- **Sémantique :**
 - Commit automatique quand le Uni émet son item
 - Rollback automatique en cas de failure dans le pipeline
 - La transaction est portée par le pipeline réactif, pas par le thread
- En test : **@TestReactiveTransaction** — transaction + rollback automatique en fin de test



@WithTransaction en pratique

```
import io.quarkus.hibernate.reactive.panache.common.WithTransaction;

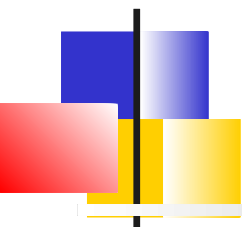
@ApplicationScoped
public class PersonService {

    @Inject PersonRepository repo;

    @WithTransaction // une transaction par appel
    public Uni<Person> create(Person p) {
        return repo.persist(p)
            .onFailure(PersistenceException.class)
            .invoke(e -> log.errorf(e, "Echec persistance: %s", p.name));
    }

    @WithTransaction
    public Uni<Void> renameAll(String from, String to) {
        return repo.update("name = ?1 where name = ?2", to, from)
            .replaceWithVoid();
    }

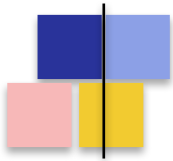
    // Sans annotation : programmatique
    public Uni<Person> createProgrammatic(Person p) {
        return Panache.withTransaction(() -> repo.persist(p));
    }
}
```

Tests d'une couche DAO réactive

Quarkus fournit du support pour écrire des tests de la couche DAO qui ne soit pas bloquant

- **@TestReactiveTransaction** : transaction réactive par test + rollback automatique.
- **UniAsserter** (paramètre de test) : enchaîner setup → action → assertions de façon non bloquante.
- **persistAndFlush()** : forcer la visibilité immédiate des inserts.



Tester la couche persistance réactive

```
import io.quarkus.test.TestReactiveTransaction;
import io.smallrye.mutiny.helpers.test.UniAssertSubscriber;

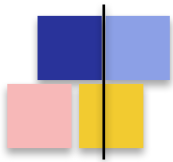
@QuarkusTest
class PersonRepositoryTest {

    @Inject PersonRepository repo;

    @Test
    @TestReactiveTransaction // tx + rollback automatique
    void create_then_find() {
        Person alice = new Person();
        alice.name = "Alice";

        repo.persist(alice)
            .chain(() -> repo.findByName("Alice"))
            .subscribe().withSubscriber(UniAssertSubscriber.create())
            .awaitItem()
            .assertCompleted()
            .assertItem(alice);
    }

    // persistAndFlush() : forcer la visibilité immédiate des inserts
    // utile quand on enchaîne plusieurs opérations sur la même entité
}
```



Alternative UniAsserter

```
import io.quarkus.test.TestReactiveTransaction;
import io.smallrye.mutiny.helpers.test.UniAssertSubscriber;

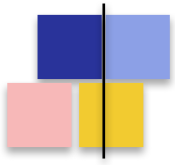
@QuarkusTest
class PersonRepositoryTest {

    @Inject PersonRepository repo;

    @Test
    @TestReactiveTransaction // tx + rollback automatique
    void create_then_find(UniAsserter asserter) {
        Person alice = new Person();
        alice.name = "Alice";

        asserter.assertThat(
            () -> repo.persist(alice).chain(() -> repo.findByName("Alice")),
            result -> {
                assertNotNull(result);
                assertEquals("Alice", result.name);
            }
        );
    }

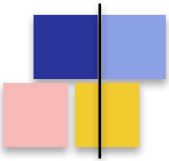
    // persistAndFlush() : forcer la visibilité immédiate des inserts
    // utile quand on enchaîne plusieurs opérations sur la même entité
}
```



Persistence

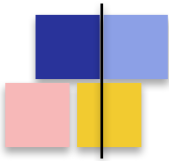
Hibernate et Panache

NoSQL



Persistence NoSQL : panorama

- Quarkus propose des **extensions réactives pour la plupart des stores NoSQL**
- **MongoDB**
 - quarkus-mongodb-client (client natif Mutiny)
 - quarkus-mongodb-panache (Active Record / Repository, comme Hibernate Panache)
- **Redis**
 - quarkus-redis-client — API réactive Mutiny
 - Idéal pour cache, sessions, compteurs distribués, rate-limiting
- **Cassandra**
 - quarkus-cassandra-client — driver DataStax avec wrapper réactif
 - Adapté aux gros volumes, séries temporelles, multi-DC
- **Elasticsearch / OpenSearch**
 - quarkus-elasticsearch-rest-client-common — moteur de recherche full-text
- **Neo4j, InfluxDB, Amazon DynamoDB** : extensions communautaires disponibles



MongoDB avec Panache Reactif

```
import io.quarkus.mongodb.panache.reactive.ReactivePanacheMongoEntity;

@MongoEntity(collection = "products")
public class Product extends ReactivePanacheMongoEntity {

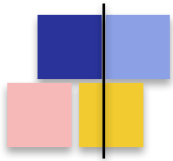
    public String name;
    public BigDecimal price;
    public List<String> tags;

    public static Uni<List<Product>> findByTag(String tag) {
        return list("tags", tag);
    }
}

// Endpoint : tout est Uni / Multi, comme Hibernate Panache
@Path("/products")
public class ProductResource {

    @GET @Path("/{id}")
    public Uni<Product> get(@RestPath ObjectId id) {
        return Product.findById(id);
    }

    @POST
    public Uni<Product> create(Product p) { return p.persist(); }
}
```



Redis pour le cache et les compteurs

```
import io.quarkus.redis.datasource.ReactiveRedisDataSource;
import io.quarkus.redis.datasource.value.ReactiveValueCommands;

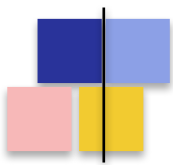
@ApplicationScoped
public class RateLimiter {

    private final ReactiveValueCommands<String, Long> counters;

    public RateLimiter(ReactiveRedisDataSource ds) {
        this.counters = ds.value(Long.class);
    }

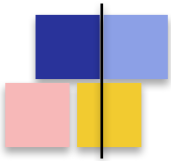
    public Uni<Boolean> tryAcquire(String userId, long limit) {
        String key = "rate:" + userId;
        return counters.incr(key)
            .chain(count ->
                count == 1
                    ? counters.getex(key, Duration.ofMinutes(1)).replaceWith(true)
                    : Uni.createFrom().item(count <= limit)
            );
    }
}

// quarkus.redis.hosts=redis://localhost:6379
```



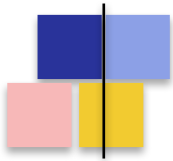
Récapitulatif des extensions

Besoin	Extension Quarkus	Style d'API
ORM relationnel réactif	<code>hibernate-reactive</code>	<code>Mutiny.SessionFactory</code>
ORM + Active Record	<code>hibernate-reactive-panache</code>	<code>Entity.findById()</code> → <code>Uni</code>
Driver SQL réactif	<code>reactive-pg-client</code> / <code>mysql</code> / <code>mssql</code> / <code>db2</code>	<code>Pool</code> , <code>Tuple</code> , <code>RowSet</code>
MongoDB documentaire	<code>mongodb-panache (reactive)</code>	<code>ReactivePanacheMongoEntity</code>
Cache, compteurs, pub/sub	<code>redis-client</code>	<code>ReactiveRedisDataSource</code>
Big data, séries temporelles	<code>cassandra-client</code>	<code>QuarkusCqlSession</code> (Mutiny)
Recherche full-text	<code>elasticsearch-rest-client</code>	<code>RestClient</code> (REST réactif)



Synthèse du module

- Hibernate Reactive : **ORM 100 % non-bloquant**, basé sur Mutiny.SessionFactory et les clients SQL réactifs Vert.x
- Panache Reactif : **Active Record (PanacheEntity)** ou **Repository (PanacheRepository)**, tout retourne Uni/Multi
- Transactions : **@WithTransaction** remplace *@Transactional* (commit/rollback porté par le pipeline réactif)
- Tests : **@TestReactiveTransaction** + UniAssertSubscriber + persistAndFlush()
- NoSQL : **MongoDB (avec Panache), Redis, Cassandra, Elasticsearch** — tous proposent une API Mutiny
- Suite : ***l'échange asynchrone entre services avec SmallRye Reactive Messaging et Kafka***



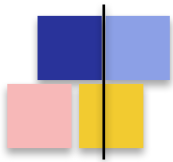
Quarkus REST

Introduction

Détection automatique et annotations

Bean Validation, Sérialisation

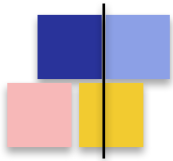
SSE



L'extension quarkus-rest

- Pour les applications RESTful, Quarkus utilise **JAX-RS (Jakarta RESTful Web Services 3.1)** via l'extension **quarkus-rest**
- Anciennement *quarkus-resteasy-reactive*, réécriture moderne et build-time de RESTEasy
- **Caractéristiques clés :**
 - Construit sur Eclipse Vert.x (modèle event-loop par défaut)
 - Supporte les 3 modèles d'exécution : impératif, réactif, virtual threads
 - Traitement des annotations à la compilation : démarrage rapide, faible empreinte mémoire
 - Sérialisation JSON via Jackson (par défaut) ou JSON-B
- **Ajout au projet :**

```
./mvnw quarkus:add-extension -Dextensions="rest,rest-jackson"
```



Une première ressource REST

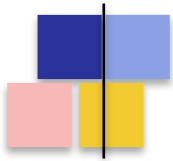
```
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/hello")
public class GreetingResource {

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String hello() {
        return "Hello from Quarkus REST";
    }
}
```

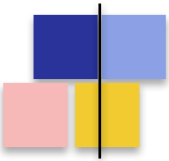
Aucune classe Application à définir, **aucun web.xml** — Quarkus scanne automatiquement les classes annotées au build.

Toutes les ressources JAX-RS sont traitées comme des **beans CDI singletons**.



Les annotations JAX-RS de base

- **Annotations de méthode HTTP :**
 - @GET, @POST, @PUT, @DELETE, @PATCH, @HEAD, @OPTIONS
- **Annotations de chemin :**
 - @Path sur la classe (préfixe) et sur la méthode (suffixe)
 - Variables : @Path("/users/{id}") avec @PathParam("id")
- **Annotations de contenu :**
 - @Produces — type MIME de la réponse (application/json, text/plain...)
 - @Consumes — type MIME accepté en entrée
- **Paramètres :**
 - @PathParam, @QueryParam, @HeaderParam, @CookieParam, @FormParam
 - Tout paramètre sans annotation est traité comme corps de requête



Une ressource CRUD complète

```
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class OrderResource {

    @Inject OrderService service;

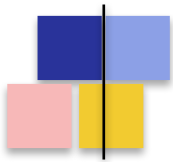
    @GET
    public List<Order> list(@QueryParam("status") String status) { ... }

    @GET
    @Path("/{id}")
    public Order get(@PathParam("id") Long id) { ... }

    @POST
    public Response create(Order order) {
        Order saved = service.save(order);
        return Response.created(URI.create("/orders/" + saved.getId())).build();
    }

    @PUT @Path("/{id}")
    public Order update(@PathParam("id") Long id, Order order) { ... }

    @DELETE @Path("/{id}")
    public void delete(@PathParam("id") Long id) { ... }
}
```



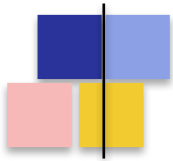
Quarkus REST

Introduction

Détection automatique et annotations

Bean Validation, Sérialisation

SSE



Détection automatique du modèle

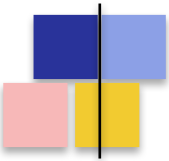
- Quarkus REST **détermine le modèle d'exécution à partir du type de retour**

```
// Retour reactif -> execution sur event-loop (non-bloquant)
@GET @Path("/{id}")
public Uni<Order> get(@PathParam("id") Long id) {
    return service.findById(id);
}

// Retour de flux -> execution sur event-loop avec back-pressure
@GET @Produces(MediaType.SERVER_SENT_EVENTS)
public Multi<Event> stream() {
    return service.events();
}

// Retour 'simple' -> execution sur worker thread (bloquant)
@GET @Path("/{id}/details")
public OrderDetails details(@PathParam("id") Long id) {
    return service.detailsBlocking(id); // JDBC, Hibernate ORM...
}

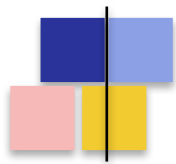
// Forcer un comportement spécifique
@GET @Path("/heavy") @Blocking
public Uni<Report> heavy() { return ...; } // sur worker malgré le Uni
```

Annotations @Rest* simplifiées

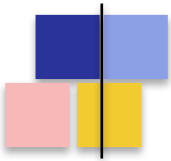
- Quarkus REST fournit une famille d'annotations **plus concises que JAX-RS**
- Le nom de la variable suffit, pas besoin de répéter le nom du paramètre

```
// Variantes : @RestPath, @RestQuery, @RestHeader,  
//             @RestCookie, @RestForm, @RestMatrix  
  
@Path("/cheeses/{type}")  
@POST  
public String allParams(  
    @RestPath String type,           // au lieu de @PathParam("type")  
    @RestMatrix String variant,  
    @RestQuery String age,           // au lieu de @QueryParam("age")  
    @RestCookie String level,  
    @RestHeader("X-Cheese-Secret") String secret,  
    @RestForm String smell  
) {  
    return type + "/" + variant + "/" + age + "/" + level  
        + "/" + secret + "/" + smell;  
}  
  
// Requete recue :  
// POST /cheeses;variant=goat/tomme?age=matured  
// Content-Type: application/x-www-form-urlencoded  
// Cookie: level=hardcore
```



Corps de requête et de réponse

- **Corps de requête** : tout paramètre sans annotation reçoit le corps
 - Types acceptés : POJO (JSON), String, byte[], InputStream, File...
 - Conversion automatique JSON ↔ objet via Jackson
- **Type de retour** : sérialisé automatiquement selon @Produces
 - POJO → JSON, byte[] → octet-stream, String → text/plain...
 - Types réactifs : Uni<T>, Multi<T>, CompletionStage<T>
 - Types fichiers : Path, FilePart, AsyncFile (streaming Vert.x)
- **Pour contrôler finement le statut HTTP** : *jakarta.ws.rs.core.Response*
 - Response.ok(entity).header(...).build()
 - Response.created(uri).build() / Response.noContent().build()
 - RestResponse<T> : variante générique typée (Quarkus REST)



Exemple réactif avec Uni

```
@Path("/users")
@Produces(MediaType.APPLICATION_JSON)
public class UserResource {

    @Inject UserService service;          // retourne des Uni / Multi

    @GET
    @Path("/{id}")
    public Uni<UserDto> get(@RestPath Long id) {
        return service.findById(id).map(this::toDto);
    }

    @POST
    public Uni<RestResponse<Void>> create(@Valid CreateUserDto dto) {
        return service.create(dto)
            .onItem().transform(user ->
                RestResponse.created(URI.create("/users/" + user.getId())));
    }
}
```



Mapping d'exceptions personnalisées

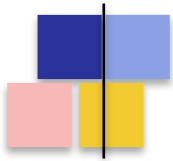
- Quarkus REST permet de **transformer toute exception métier en réponse HTTP**
- Deux approches : `@Provider + ExceptionMapper`, ou `@ServerExceptionMapper`

```
// Approche moderne (Quarkus REST) : @ServerExceptionMapper
public class GlobalExceptionMappers {

    @ServerExceptionMapper
    public RestResponse<ErrorDto> handleNotFound(NotFoundException ex) {
        return RestResponse.status(404,
            new ErrorDto("NOT_FOUND", ex.getMessage()));
    }

    @ServerExceptionMapper
    public RestResponse<ErrorDto> handleValidation(ConstraintViolationException ex) {
        List<String> errors = ex.getConstraintViolations().stream()
            .map(v -> v.getPropertyPath() + " : " + v.getMessage())
            .toList();
        return RestResponse.status(400, new ErrorDto("INVALID", errors));
    }
}

// Approche historique : implements ExceptionMapper<E>
```



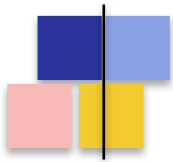
Quarkus REST

Introduction

Détection automatique et annotations

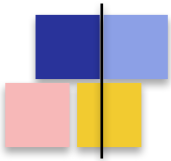
Bean Validation, Sérialisation

SSE



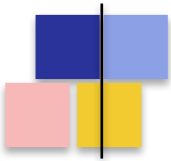
Bean Validation

- Standard **Jakarta Bean Validation** — annotations déclaratives pour valider les objets
- Extension : **quarkus-hibernate-validator**
- **Annotations courantes :**
 - @NotNull, @NotBlank, @NotEmpty — présence et non-vide
 - @Size(min, max), @Min, @Max — bornes
 - @Pattern(regexp = ...) — expression régulière
 - @Email, @Past, @Future — formats classiques
 - @Valid — déclenche la validation en cascade sur un objet imbriqué
- Côté endpoint : **@Valid sur le paramètre, validation auto avant l'appel à la méthode**
 - Si invalide → ConstraintViolationException → réponse 400 par défaut



Bean Validation en pratique

```
public record CreateUserDto(  
    @NotBlank @Size(min = 3, max = 50)  
    String username,  
  
    @NotBlank @Email  
    String email,  
  
    @NotBlank @Pattern(regexp = "^(?=.*[A-Z])(?=.*\\d){8,}$",  
        message = "Doit contenir 1 majuscule, 1 chiffre, min 8 caracteres")  
    String password,  
  
    @Min(18) @Max(120)  
    int age  
) {}  
  
@Path("/users")  
public class UserResource {  
    @POST  
    public Uni<User> create(@Valid CreateUserDto dto) {  
        return userService.create(dto);  
        // @Valid declenche la validation AVANT l'execution de la methode  
        // En cas d'echec : 400 + corps decrivant les violations  
    }  
}
```

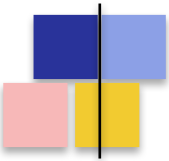


Sérialisation JSON

- Quarkus REST utilise **Jackson** (par défaut) ou **JSON-B** (Yasson)
- Activé via l'extension quarkus-rest-jackson ou quarkus-rest-jsonb
- **Configuration globale Jackson :**

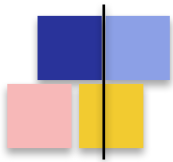
```
# application.properties
quarkus.jackson.fail-on-unknown-properties=false
quarkus.jackson.write-dates-as-timestamps=false
quarkus.jackson.serialization-inclusion=non-null
quarkus.jackson.timezone=Europe/Paris

# Equivalent par customizer programmatique
@Singleton
public class JacksonConfig implements ObjectMapperCustomizer {
    public void customize(ObjectMapper mapper) {
        mapper.registerModule(new JavaTimeModule());
        mapper.setSerializationInclusion(Include.NON_NULL);
    }
}
```

Annotations Jackson courantes

```
public record OrderDto(  
    Long id,  
  
    @JsonProperty("customer_id")  
    Long customerId,                // renomme dans le JSON  
  
    @JsonFormat(pattern = "yyyy-MM-dd")  
    LocalDate orderDate,  
  
    @JsonIgnore  
    String internalNotes,          // jamais serialise  
  
    @JsonInclude(JsonInclude.Include.NON_EMPTY)  
    List<String> tags,              // omis si vide  
  
    BigDecimal amount  
) {  
  
    @JsonIgnore  
    public boolean isExpensive() {    // methode non serialisee  
        return amount.compareTo(BigDecimal.valueOf(1000)) > 0;  
    }  
}
```



Quarkus REST

Introduction

Détection automatique et annotations

Bean Validation, Sérialisation, Sécurité

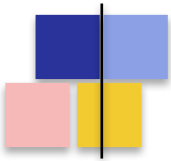
SSE



Server-Sent Events : qu'est-ce que c'est ?

- Standard W3C, **flux d'événements unidirectionnel serveur** → **client** sur HTTP
- Une seule connexion HTTP ouverte, le serveur pousse des messages au fil de l'eau
- **Quand l'utiliser ?**
 - Tableaux de bord en temps réel, notifications push
 - Streaming de logs, flux de prix, mises à jour de statut
 - Cas où WebSocket serait excessif (pas de communication bidirectionnelle)
- **Comparaison rapide :**
 - SSE : simple, sur HTTP, reconnexion auto côté navigateur
 - WebSocket : bidirectionnel, plus complexe, négociation Upgrade
 - Long-polling : ancien, coûteux, à éviter

Côté Quarkus : **un endpoint qui retourne un Multi<T> avec @Produces(SERVER_SENT_EVENTS)**



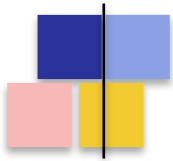
Un endpoint SSE basique

```
@Path("/ticks")
public class TickResource {

    @GET
    @Produces(MediaType.SERVER_SENT_EVENTS)
    public Multi<Long> ticks() {
        return Multi.createFrom()
            .ticks().every(Duration.ofSeconds(1))
            .onItem().transform(n -> n + 1);
    }
}

// Cote client (JavaScript)
// const evt = new EventSource('/ticks');
// evt.onmessage = e => console.log('tick:', e.data);

// Cote curl
// $ curl -N http://localhost:8080/ticks
// data:1
// data:2
// data:3
// ...
```



SSE avec événements structurés

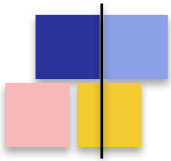
- Avec **@RestStreamElementType** on précise le format de chaque événement (JSON unique)

```
@Path("/orders")
public class OrderResource {

    @Inject OrderService service;

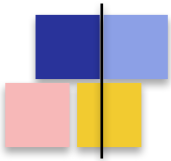
    @GET
    @Path("/stream")
    @Produces(MediaType.SERVER_SENT_EVENTS)
    @RestStreamElementType(MediaType.APPLICATION_JSON)
    public Multi<OrderEvent> stream() {
        return service.subscribe() // Multi<OrderEvent>
            .onItem().invoke(e -> log.infoof("Publishing: %s", e.id()))
            .onCompletion().invoke(() -> log.info("Stream closed"));
    }
}

// Output cote client
// data:{"id":"o-1","status":"PAID","amount":42.50}
// data:{"id":"o-2","status":"SHIPPED","amount":18.00}
// ...
```



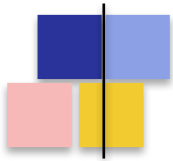
Bonnes pratiques SSE

- Toujours retourner un **Multi<T>**, jamais une List ou un Stream
 - Un Multi est paresseux et respecte la back-pressure ; une List est chargée en mémoire
- **Gérer proprement la fermeture de connexion**
 - `onCancellation()` : appelé si le client ferme l'EventSource
 - `onCompletion()` / `onFailure()` : pour libérer les ressources (BDD, Kafka...)
- **Limiter le débit côté serveur**
 - Si la source pousse trop vite : `.onOverflow().drop()` ou `.throttle()`
 - Évite de saturer la mémoire si un client est lent
- **Pour les clients qui peuvent se déconnecter / reconnecter**
 - Envoyer un identifiant d'événement (Last-Event-ID) pour la reprise
 - Inclure un ping périodique pour détecter les connexions zombies



Synthèse du module

- L'extension **quarkus-rest** implémente JAX-RS au-dessus de Vert.x
- Modèle d'exécution **détection automatique** selon le type de retour (Uni / Multi / type simple)
- Annotations **@Rest*** simplifient la lecture (le nom de la variable suffit)
- SSE : **Multi<T> + @Produces(SERVER_SENT_EVENTS) + @RestStreamElementType**
- Bean Validation : **@Valid** sur les DTO, mapping d'erreurs personnalisable
- Sérialisation : **Jackson** par défaut, configuration globale ou par annotation
- Sécurité : **@RolesAllowed + JWT/OIDC, SecurityIdentity** pour le contexte
- Suite : ***orchestrer plusieurs services REST entre eux (RestClient, Fault Tolerance)***

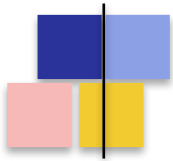


Couche Service

Composition de service

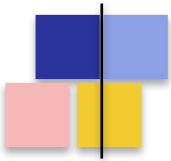
Rest Client

Fault Tolerance



Pourquoi une couche service ?

- La ressource REST **ne devrait contenir aucune logique métier**
 - Elle traduit HTTP ↔ objets et délègue à un service
 - Elle gère le statut HTTP, les codes d'erreur, la sérialisation
- **La couche service porte :**
 - La logique métier (règles, calculs, validations applicatives)
 - L'orchestration entre plusieurs sources (BDD, autres services, cache, broker)
 - La gestion des transactions
 - La résilience (retry, fallback, timeout)
- *Dans Quarkus : **beans CDI annotés @ApplicationScoped (par défaut)***
 - Injection par @Inject dans les ressources et entre services
 - Cycle de vie géré par le conteneur, pas de new manuel



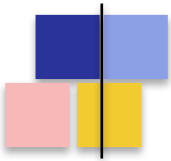
Un service réactif typique

```
import jakarta.enterprise.context.ApplicationScoped;
import io.smallrye.mutiny.Uni;

@ApplicationScoped
public class OrderService {

    @Inject OrderRepository repo;
    @Inject InventoryClient inventory;          // RestClient externe
    @Inject NotificationService notifier;

    public Uni<Order> placeOrder(CreateOrderDto dto) {
        return inventory.check(dto.productId(), dto.quantity())
            .onItem().transformToUni(ok ->
                ok ? repo.save(Order.from(dto))
                  : Uni.createFrom().failure(new OutOfStockException()))
            .onItem().call(order -> notifier.sendConfirmation(order))
            .onItem().invoke(order ->
                log.infof("Order placed: %s", order.getId()));
    }
}
```

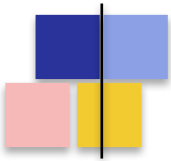


Composition séquentielle

- Cas typique : **le résultat d'un appel sert d'entrée au suivant**
 - → utiliser `chain()` / `transformToUni()` — vu dans le module Mutiny

```
// Recuperer le profil utilisateur, puis ses preferences, puis ses recommandations
public Uni<Recommendations> getRecommendations(Long userId) {
    return userService.findById(userId)
        .chain(user -> prefService.loadPreferences(user.getId()))
        .map(prefs -> new UserContext(user, prefs))
        .chain(ctx -> recoEngine.compute(ctx)
            .map(items -> new Recommendations(ctx.user(), items)));
}

// Variante : enchaîner sans transporter le contexte intermediaire
public Uni<Void> processOrder(Long orderId) {
    return orderRepo.findById(orderId)
        .chain(this::validateOrder)           // Uni<Order>
        .chain(this::reserveStock)           // Uni<Order>
        .chain(this::chargePayment)          // Uni<Order>
        .chain(this::shipOrder)              // Uni<Void>
        .onFailure().recoverWithUni(this::compensate);
}
```



Composition parallèle

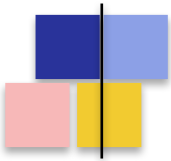
- Quand les appels sont **indépendants entre eux**, les lancer en parallèle réduit la latence
 - → `Uni.combine()` — vu dans le module Mutiny

```
// Agreger un profil depuis 3 services en parallele
public Uni<UserProfile> buildProfile(Long userId) {
    Uni<User> userUni = userService.findById(userId);
    Uni<List<Order>> ordersUni = orderService.findByUser(userId);
    Uni<Loyalty> loyaltyUni = loyaltyService.getStatus(userId);

    return Uni.combine().all().unis(userUni, ordersUni, loyaltyUni)
        .with((user, orders, loyalty) ->
            new UserProfile(user, orders, loyalty));
}

// Tolerance partielle : si un appel echoue, on continue
public Uni<UserProfile> buildProfileTolerant(Long userId) {
    Uni<List<Order>> ordersUni = orderService.findByUser(userId)
        .onFailure().recoverWithItem(List.of());
    Uni<Loyalty> loyaltyUni = loyaltyService.getStatus(userId)
        .onFailure().recoverWithItem(Loyalty.none());

    return Uni.combine().all().unis(
        userService.findById(userId), ordersUni, loyaltyUni
    ).with(UserProfile::new);
}
```



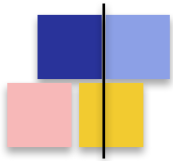
Composition avec un Multi

- Pattern courant : **pour chaque item d'un flux, faire un appel asynchrone**

```
// Pour chaque commande, recuperer son detail (parallelisme controle)
public Multi<OrderDetail> loadOrderDetails() {
    return orderRepo.streamRecent()                // Multi<Order>
        .onItem().transformToUni(order ->
            detailService.fetch(order.getId())      // Uni<OrderDetail>
        )
        .merge(8);                                // 8 appels en parallele max

    // .concatenate() : sequentiel (preserve l'ordre)
    // .merge(n)       : parallele avec borne (ordre non garanti)
}

// Eclater chaque commande en plusieurs items
public Multi<LineItem> allLineItems() {
    return orderRepo.streamRecent()
        .onItem().transformToMulti(order ->
            Multi.createFrom().iterable(order.getLineItems())
        ).concatenate();
}
```

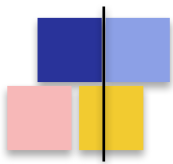


Couche Service

Composition de service

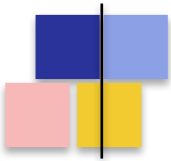
Rest Client

Fault Tolerance



MicroProfile Rest Client

- Standard **Eclipse MicroProfile** pour consommer des APIs REST de manière déclarative
- Implémentation dans Quarkus : extension quarkus-rest-client
- **Principe** : on définit une **interface Java** qui décrit l'API distante, Quarkus génère l'implémentation
- **Avantages** :
 - Mêmes annotations JAX-RS que côté serveur : familier, peu de plomberie
 - Typage fort : les DTO d'entrée et de retour sont partagés
 - Configuration externalisée : URL, timeout, etc. dans application.properties
 - Compatible avec Fault Tolerance : @Retry, @Timeout, @CircuitBreaker
 - Versions sync et réactive (Uni / Multi) selon le type de retour
- Ajout : `./mvnw quarkus:add-extension -Dextensions="rest-client,rest-client-jackson"`



Définir un Rest Client

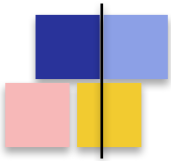
```
import io.smallrye.mutiny.Uni;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;

@RegisterRestClient(configKey = "inventory-api")
@Path("/inventory")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public interface InventoryClient {

    @GET
    @Path("/products/{id}/stock")
    Uni<StockInfo> getStock(@PathParam("id") Long productId);

    @POST
    @Path("/products/{id}/reserve")
    Uni<Reservation> reserve(@PathParam("id") Long productId,
                           ReservationRequest request);

    @GET
    @Path("/products")
    Multi<Product> streamCatalog();           // SSE ou JSON streaming
}
```

Configuration du Rest Client

- Le **configKey** déclaré dans *@RegisterRestClient* fait le pont vers la configuration externe

```
# application.properties

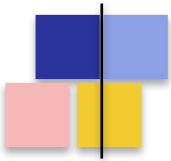
# URL de base du service distant
quarkus.rest-client.inventory-api.url=https://inventory.example.com

# Timeouts (en ms)
quarkus.rest-client.inventory-api.connect-timeout=2000
quarkus.rest-client.inventory-api.read-timeout=5000

# Logging des requetes / reponses (dev)
quarkus.rest-client.inventory-api.scope=jakarta.inject.Singleton
quarkus.rest-client.logging.scope=request-response
quarkus.rest-client.logging.body-limit=1024

# Authentification : header automatique sur tous les appels
quarkus.rest-client.inventory-api.headers.Authorization=Bearer ${INVENTORY_TOKEN}

# TLS / proxy / compression
quarkus.rest-client.inventory-api.trust-store=classpath:truststore.jks
quarkus.rest-client.inventory-api.proxy-address=proxy.corp:8080
```

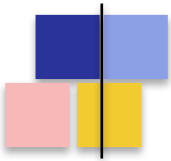


Utiliser un Rest Client

```
@ApplicationScoped
public class OrderService {

    @Inject
    @RestClient                               // injection du client
    InventoryClient inventoryClient;

    public Uni<Order> placeOrder(CreateOrderDto dto) {
        return inventoryClient.getStock(dto.productId())
            .onItem().transformToUni(stock -> {
                if (stock.available() < dto.quantity()) {
                    return Uni.createFrom().failure(new OutOfStockException());
                }
                return inventoryClient.reserve(
                    dto.productId(),
                    new ReservationRequest(dto.quantity())
                );
            })
            .onItem().transformToUni(reservation ->
                orderRepo.save(Order.from(dto, reservation))
            );
    }
}
```



Construction programmatique

- Quand l'URL est dynamique (multi-tenant, discovery), on utilise **RestClientBuilder**

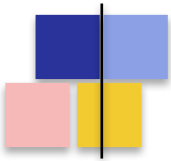
```
@ApplicationScoped
public class TenantInventoryService {

    public Uni<StockInfo> getStock(String tenant, Long productId) {

        InventoryClient client = QuarkusRestClientBuilder.newBuilder()
            .baseUrl(URI.create("https://" + tenant + ".inventory.example.com"))
            .connectTimeout(2, TimeUnit.SECONDS)
            .readTimeout(5, TimeUnit.SECONDS)
            .header("X-Tenant", tenant)
            .build(InventoryClient.class);

        return client.getStock(productId);
    }
}

// Variante MicroProfile standard : RestClientBuilder.newBuilder()
// Variante Quarkus (plus de fonctionnalites) : QuarkusRestClientBuilder
```



Filtres : intercepter les appels

- Pour ajouter **des headers dynamiques, du logging, du tracing**, on utilise des filtres

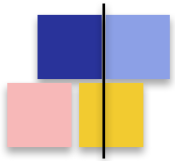
```
// Filtre cote client : ajoute un header de traçage a chaque requete
@Provider
public class TracingFilter implements ClientRequestFilter {

    @Inject TracingContext tracing;

    @Override
    public void filter(ClientRequestContext ctx) throws IOException {
        ctx.getHeaders().add("X-Trace-Id", tracing.currentTraceId());
        ctx.getHeaders().add("X-Span-Id", tracing.currentSpanId());
    }
}

// Application a un Rest Client specifique
@registerRestClient(configKey = "inventory-api")
@registerProvider(TracingFilter.class)
public interface InventoryClient { ... }

// Variante : @ClientHeaderParam pour un header simple, sans creer de filtre
// @ClientHeaderParam(name="Authorization", value="Bearer {token}")
```

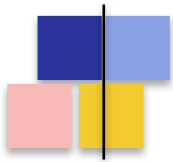


Couche Service

Composition de service

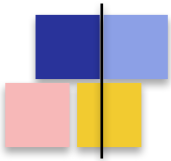
Rest Client

Fault Tolerance



SmallRye Fault Tolerance

- Implémentation de **MicroProfile Fault Tolerance** fournie par Quarkus
- Extension : quarkus-smallrye-fault-tolerance
- **Les 5 patterns de résilience standard :**
 - – **@Timeout** — coupe l'appel après N millisecondes
 - – **@Retry** — relance N fois en cas d'échec
 - – **@Fallback** — résultat de secours si tout échoue
 - – **@CircuitBreaker** — interrompt les appels vers un service défaillant
 - – **@Bulkhead** — limite le nombre d'appels concurrents
- Applicables sur **tout bean CDI ou Rest Client**, combinables entre elles



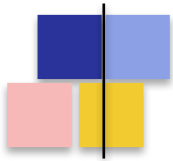
@Timeout et @Retry

```
@ApplicationScoped
public class WeatherService {

    @Inject @RestClient WeatherClient client;

    // Echoue si pas de reponse en 2 secondes
    @Timeout(2000)
    public Uni<Weather> get(String city) {
        return client.fetch(city);
    }

    // 3 tentatives, delai initial 200 ms, backoff exponentiel jusqu'a 5s
    @Retry(maxRetries = 3,
        delay = 200, delayUnit = ChronoUnit.MILLIS,
        maxDuration = 5000, maxDurationUnit = ChronoUnit.MILLIS,
        jitter = 100,
        retryOn = { IOException.class, WeatherTransientException.class },
        abortOn = { WeatherInvalidCityException.class })
    public Uni<Weather> getReliable(String city) {
        return client.fetch(city);
    }
}
```



@Fallback : la roue de secours

- Définit un **comportement de repli** quand l'appel échoue, après tous les retries

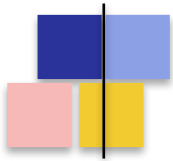
```
@ApplicationScoped
public class PricingService {

    @Inject @RestClient PricingClient client;

    // Methode de fallback dans la meme classe
    @Retry(maxRetries = 2)
    @Fallback(fallbackMethod = "defaultPrice")
    public Uni<Price> getPrice(Long productId) {
        return client.fetchPrice(productId);
    }

    Uni<Price> defaultPrice(Long productId) {           // meme signature
        return Uni.createFrom().item(Price.defaultFor(productId));
    }

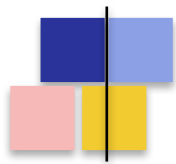
    // Variante : classe FallbackHandler dediee (utile pour Rest Client)
    @Fallback(PricingFallbackHandler.class)
    public Uni<Price> getPriceV2(Long productId) { ... }
}
```

@CircuitBreaker : couper court

- Quand un service échoue trop souvent, **arrêter de l'appeler pendant un temps**
- Trois états : CLOSED (normal) → OPEN (rejet immédiat) → HALF_OPEN (test)

```
@CircuitBreaker(  
    requestVolumeThreshold = 10,    // taille de la fenetre glissante  
    failureRatio = 0.5,            // >= 50% d'echecs -> ouverture  
    delay = 5000,                  // 5 secondes avant de tester  
    successThreshold = 2           // 2 succes consecutifs en HALF_OPEN -> CLOSED  
)  
@Retry(maxRetries = 2)  
@Fallback(fallbackMethod = "cached")  
public Uni<Price> getPrice(Long id) {  
    return client.fetchPrice(id);  
}  
  
Uni<Price> cached(Long id) {  
    return cache.get(id);           // valeur potentiellement perimee  
}  
  
// Sequence : 10 appels -> 5+ echecs -> CIRCUIT OPEN  
// Les 5 prochaines secondes : echec immediat + fallback  
// Apres : 1 appel passe pour tester (HALF_OPEN)  
// Si OK 2 fois : retour CLOSED. Sinon : reouverture.
```



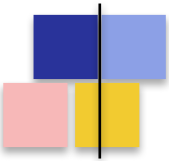
@Bulkhead : limiter la concurrence

- Limite le **nombre d'invocations concurrentes** d'une méthode
- Évite qu'un appel coûteux ne sature toutes les ressources
- Métaphore : compartiments étanches d'un navire — un compartiment qui coule ne fait pas couler le navire

```
// Max 10 appels concurrents a ce service
@Bulkhead(10)
public Uni<Report> heavyReport(Long id) {
    return reportService.generate(id);
}

// Variante asynchrone : file d'attente de 5 en plus des 10 en cours
@Bulkhead(value = 10, waitingTaskQueue = 5)
public Uni<Report> reportAsync(Long id) {
    return reportService.generate(id);
}

// Combinaison classique : Bulkhead + Timeout + Fallback
@Bulkhead(20)
@Timeout(3000)
@Fallback(fallbackMethod = "degraded")
public Uni<Recommendations> recommend(Long userId) {
    return recoClient.compute(userId);
}
```



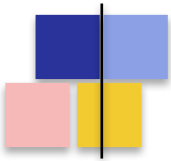
Combinaison sur un Rest Client

- Les annotations Fault Tolerance peuvent être **placées directement sur l'interface Rest Client**

```
@RegisterRestClient(configKey = "fruit-api")
@Path("/api/fruit")
public interface FruityViceService {

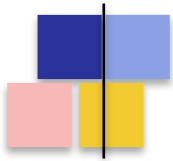
    @GET
    @Path("/{name}")
    @Produces(MediaType.APPLICATION_JSON)
    @Timeout(2000)
    @Retry(maxRetries = 3, delay = 500)
    @CircuitBreaker(requestVolumeThreshold = 4, failureRatio = 0.75, delay = 10000)
    @Fallback(FruityViceFallback.class)
    Uni<FruityVice> getFruitByName(@PathParam("name") String name);

    class FruityViceFallback implements FallbackHandler<Uni<FruityVice>> {
        @Override
        public Uni<FruityVice> handle(ExecutionContext context) {
            return Uni.createFrom().item(FruityVice.empty());
        }
    }
}
```



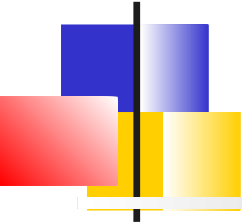
Bonnes pratiques résilience

- **Configuration externalisée** : éviter les valeurs en dur dans les annotations
 - `io.smallrye.faulttolerance.inventory-api/getStock/Retry/maxRetries=5`
 - Format MicroProfile : `<class>/<method>/<annotation>/<param>`
- **Ordre des opérations** (à connaître) :
 - Fallback ← Retry ← CircuitBreaker ← Timeout ← Bulkhead ← appel
 - Le Timeout protège chaque tentative, le Retry les enchaîne
- **Distinguer les erreurs transitoires des erreurs définitives**
 - `retryOn` / `abortOn` : ne pas retry un 404 ou une `ConstraintViolation`
- **Métriques natives** : observer avant d'ajuster
 - `ft.invocations.total`, `ft.timeout.calls.total`, `ft.circuitbreaker.calls.total`...
 - Exposées via `/q/metrics` avec `quarkus-micrometer-registry-prometheus`



Synthèse du module

- La couche service **porte la logique métier et l'orchestration** (beans CDI @ApplicationScoped)
- Composition : **chain pour le séquentiel, combine pour le parallèle, merge(n) pour le streaming**
- Rest Client : **interface Java + @RegisterRestClient**, config externalisée, typage fort, version réactive
- 5 annotations Fault Tolerance : **@Timeout, @Retry, @Fallback, @CircuitBreaker, @Bulkhead**
- Applicables sur **bean CDI ou Rest Client**, combinables, configurables via properties
- Suite : ***la persistance réactive avec Hibernate Reactive et Panache***

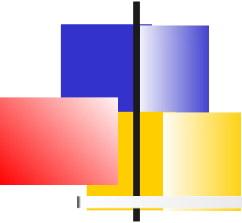


Messaging

Smallrye Reactive Messaging

Connecteur Kafka

Ack, Erreurs, Stratégies de commit
Tests

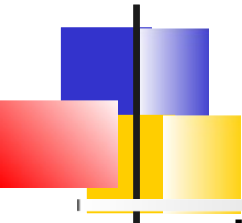


Quarkus Messaging

Quarkus Messaging est une API qui supporte différents brokers (*Apache Kafka, AMQP, Apache Camel, JMS, MQTT, etc.*)

C'est une implémentation de **Eclipse MicroProfile Reactive Messaging 2.0** :

- Les applications échangent des **messages** contenant la payload et des méta-données.
- Les messages transitent dans des **canaux** (channels). Les applications se connectent aux canaux.
- Les canaux sont connectés aux système de messagerie sous-jacent via des **connecteurs (connector)**.
Chaque connecteur est dédié à une technologie de messagerie spécifique et sont packagés dans des extensions



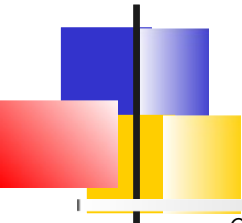
Réception de message avec Reactive Messaging

La réception de message peut s'effectuer en annotant une méthode avec **@Incoming** et en spécifiant le nom du canal.

La méthode peut récupérer via son argument :

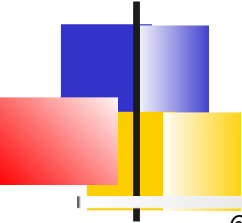
- La charge utile (payload)
- Le message complet avec `Message`
- Le record avec `ConsumerRecord` ou `Record`

Il est également possible de se faire injecter un *Multi* représentant le stream de message



Exemple

```
@ApplicationScoped
public class PriceConsumer {
    @Incoming("prices")
    public void consume(double price) { // Juste la charge utile }
    @Incoming("prices")
    public CompletionStage<Void> consume(Message<Double> msg) {
        var metadata = msg.getMetadata(IncomingKafkaRecordMetadata.class).orElseThrow();
        double price = msg.getPayload();
        // Ack manuel (On indique à Quarkus que le msg a été consommé)
        return msg.ack();
    }
    @Incoming("prices")
    public void consume(ConsumerRecord<String, Double> record) { // API Kafka
        String key = record.key();
        String value = record.value();
        String topic = record.topic();
        int partition = record.partition();
    }
    @Incoming("prices")
    public void consume(Record<String, Double> record) { // Connecteur Kafka
        String key = record.key();
        String value = record.value();
    }
}
```



Exemple : Injection de channel

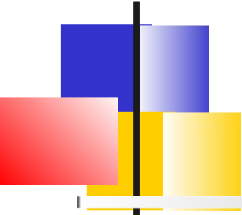
```
@Path("/prices")
public class PriceResource {

    @Inject
    @Channel("prices")
    Multi<Double> prices;

    @GET
    @Path("/prices")
    @RestStreamElementType(MediaType.TEXT_PLAIN) // produit MediaType.SERVER_SENT_EVENTS
    public Multi<Double> stream() {
        return prices;
    }
}
```

Les autres types possibles sont :

```
@Inject @Channel("prices") Multi<Double> streamOfPayloads;
@Inject @Channel("prices") Multi<Message<Double>> streamOfMessages;
@Inject @Channel("prices") Publisher<Double> publisherOfPayloads;
@Inject @Channel("prices") Publisher<Message<Double>> publisherOfMessages;
```



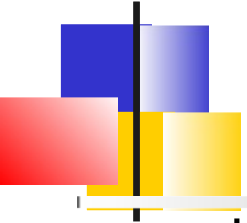
Envoi de message

Via l'annotation **@Outgoing**, une méthode peut publier un message vers un *topic*

```
@ApplicationScoped
public class KafkaPriceProducer {

    private final Random random = new Random();

    @Outgoing("prices-out")
    public Multi<Double> generate() {
        // Build an infinite stream of random prices
        // It emits a price every second
        return Multi.createFrom().ticks().every(Duration.ofSeconds(1))
            .map(x -> random.nextDouble());
    }
}
```



Envoi avec Emitter

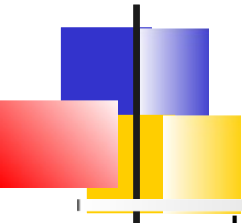
L'autre façon d'envoyer des messages est de se faire injecter par le framework un bean ***Emitter***.

L'envoi retourne un *CompletionStage*, terminé lorsque le message est acquitté.

```
@Path("/prices")
public class PriceResource {

    @Inject
    @Channel("price-create")
    Emitter<Double> priceEmitter;

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    public void addPrice(Double price) {
        // Exception si nack
        CompletionStage<Void> ack = priceEmitter.send(price);
    }
}
```



Processor

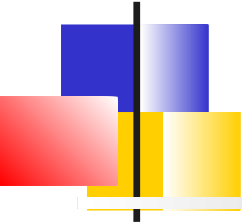
Un ***processor***¹ dans les architecture *event-driven* est un micro-service qui lit un *topic* en entrée, effectue un traitement et écrit sur un *topic* de sortie.

```
@ApplicationScoped
public class PriceProcessor {

    private static final double CONVERSION_RATE = 0.88;

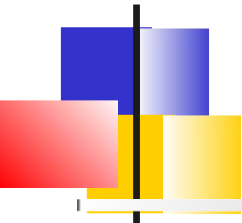
    @Incoming("price-in")
    @Outgoing("price-out")
    public double process(double price) {
        return price * CONVERSION_RATE;
    }

    // Version asynchrone
    @Incoming("price-in")
    @Outgoing("price-out")
    public Multi<Double> process(Multi<Integer> prices) {
        return prices.filter(p -> p > 100).map(p -> p * CONVERSION_RATE);
    }
}
```



Messaging

Smallrye Reactive Messaging
Connecteur Kafka
Ack, Erreurs, Stratégies de commit
Tests



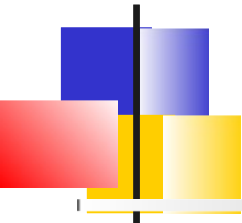
Kafka

Apache Kafka a de nombreux cas d'usage :

- Message Broker
- Bufferisation d'événements
- Architecture Event-driven
- Journal d'évènements

Ses fonctionnalités de bases sont :

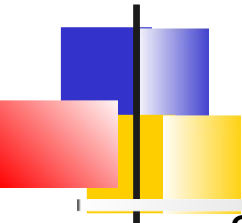
- Publier des événements vers des topics
- Stocker les évènements de manière durable et fiable dans des **topics** sous la forme de **record**.
- S'abonner aux topics. Et offrir des garanties de livraisons des messages malgré des défaillances
- S'abonner rétroactivement .



Quarkus et Kafka

Quarkus supporte Apache Kafka via :

- *SmallRye Reactive Messaging.*
quarkus-messaging-kafka
- L'API native Kafka
quarkus-kafka-client
- Kafka Streams (Traitement d'évènements via l'API Kafka Streams)
quarkus-kafka-streams
- Kafka via Camel
camel-quarkus-kafka
- Kafka via Camel sur AWS
camel-quarkus-aws2-msk



Dev Services

Si une extension liée à Kafka est présente , Dev Services démarre automatiquement un container Kafka prêt à l'emploi (profils *dev* et *test*).

Dev Services peut être configuré :

- **`quarkus.kafka.devservices.enabled`** : true/false
- Si **`kafka.bootstrap.servers`** est configuré, Dev Services n'est pas démarré
- **`quarkus.kafka.devservices.shared`** : true/false. Indique si le mécanisme de partage automatique du broker (entre application producteur et consommateur) est activé
- **`quarkus.kafka.devservices.port`** : Fixer le port d'écoute. Par défaut aléatoire
- **`quarkus.kafka.devservices.image-name`** : Le nom de l'image. Quarkus supporte Redpanda et Strimzi
- **`quarkus.kafka.devservices.topic-partitions.<topic>`** : Permet d'initialiser des topics avec un nombre de partitions



Configuration minimale Kafka

Une configuration minimale pourrait être :

```
%prod.kafka.bootstrap.servers=kafka:9092
```

```
mp.messaging.incoming.prices.connector=smallrye-kafka
```

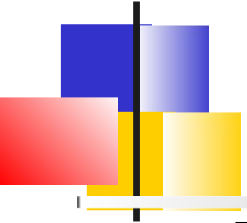
- Configure dans le profil de *prod*, l'adresse d'un broker kafka faisant partie d'un cluster. On peut indiquer plusieurs adresses
- Configuration d'un connecteur Kafka gérant le canal *prices* (donc le topic Kafka)
 - Si un seul connecteur est dans le classpath, cette configuration est optionnelle

–

Sur un canal, de nombreuses options de configuration sont disponibles selon la syntaxe :

```
mp.messaging.incoming.<nom-du-canal>.attribute=value
```

```
mp.messaging.outgoing.<nom-du-canal>.attribute=value
```



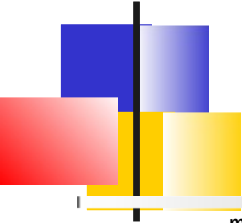
Sérialiseurs

Des sérialiseurs et des désérialiseurs sont utilisés par Kafka pour stocker les objets Java.

En utilisant l'annotation `@Channel`, on profite de configurations automatiques.

- Le nom du canal est associé à un *topic* du même nom
- Des sérialiseurs sont déduits pour les types de base et pour les classes du domaine si l'on a Jackson ou JSON-B dans le classpath

Le mécanisme d'auto-détection n'est pas applicable lors d'une sérialisation Avro et l'utilisation de registre de Schéma



Configuration typique Avro / Confluent

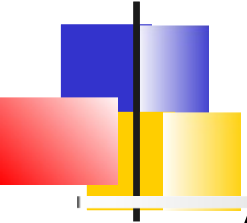
```
mp.messaging.incoming.prices.connector=smallrye-kafka  
mp.messaging.incoming.prices.topic=prices
```

```
# Désérialiseurs Avro
```

```
mp.messaging.incoming.prices.key.deserializer=io.confluent.kafka.serializers.KafkaAvroDeserializer  
mp.messaging.incoming.prices.value.deserializer=io.confluent.kafka.serializers.KafkaAvroDeserializer
```

```
# URL du Schema Registry
```

```
mp.messaging.connector.smallrye-kafka.schema.registry.url=http://localhost:8081
```



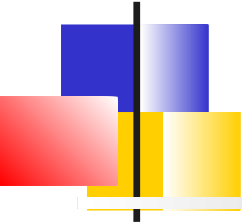
Groupe et partitions

Avec Kafka, un ***groupe*** est un ensemble de consommateurs qui coopèrent pour consommer les données d'un *topic*.

Un topic est divisé en partitions.
(Répartition des données sur les différents nœuds du cluster)

Les partitions d'un topic sont attribuées parmi les consommateurs du groupe :

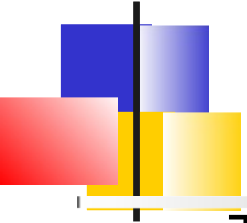
- Chaque partition est affectée à un seul consommateur d'un groupe.
- Un consommateur peut être affecté à plusieurs partitions si le nombre de partitions est supérieur au nombre de consommateurs dans le groupe.



Messaging

Smallrye Reactive Messaging
Connecteur Kafka

Ack, Erreurs, Stratégies de commit
Tests



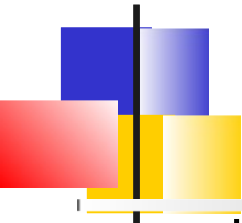
Stratégies de ack

Tous les messages reçus par un consommateur doivent être acquittés. Cela indique au framework que le traitement du message a réussi

- Si la méthode reçoit un *Record* ou la payload, le message sera acquitté au retour de la méthode
- Si la méthode renvoie un autre flux réactif ou *CompletionStage*, le message sera acquitté lorsque le message en aval sera acquitté
- Si la méthode reçoit un *Message*, l'acquittement est manuel

`msg.ack();`

On peut surcharger ce comportement par défaut via l'annotation **@Acknowledgment**



Stratégie de commit

Lorsqu'un message est acquitté, le connecteur invoque une stratégie de commit qui permet de déplacer l'offset géré par Kafka.

3 stratégies sont possibles :

- ***throttled*** : Commit à intervalle régulier.
Garantie *at-least-once* même si le canal effectue un traitement asynchrone.
- ***latest*** : Commit du dernier *ack*.
Garantie *at-least-once* si le canal effectue un traitement synchrone
- ***ignore*** : Délègue le commit au client Kafka sous-jacent.
Ne garantit pas *at-least-once*
- ***checkpoint*** (*expérimental*) : L'offset est stocké localement dans un *state store*

Attribut de configuration :

`mp.messaging.incoming.<channel>.commit-strategy`

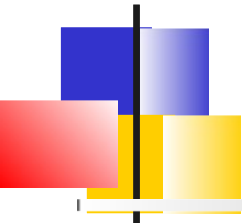


Stratégies de gestion d'erreur

Si le message n'est pas acquitté, une stratégie de gestion d'erreur indiquée par la propriété de configuration ***failure-strategy*** est appliquée

failure-strategy peut prendre 3 valeurs :

- ***fail*** : Faire échouer l'application, plus aucun enregistrement ne sera traité (stratégie par défaut).
L'offset de l'enregistrement qui n'a pas été traité correctement n'est pas committé.
- ***ignore*** : l'échec est loggé, mais le traitement continue.
L'offset de l'enregistrement qui n'a pas été traité correctement est committé.
- ***dead-letter-queue*** : l'offset de l'enregistrement qui n'a pas été traité correctement est committé, mais l'enregistrement est écrit dans un topic spécifique de Kafka.
D'autres configurations sont possible sur le topic *dead-letter*

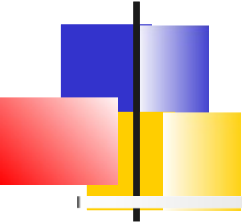


Dead-letter queue (DLQ)

- Les enregistrements en échec sont isolés dans un topic dédié plutôt que de bloquer le flux.
- L'offset du record fautif est committé : le traitement continue.
- Topic par défaut : `dead-letter-topic-<topic>` ; nom, clé/valeur et sérialiseurs configurables.

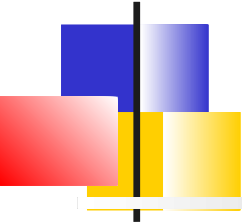
```
mp.messaging.incoming.prices.failure-strategy=dead-letter-queue
```

```
mp.messaging.incoming.prices.dead-letter-queue.topic=prices-dlq
```



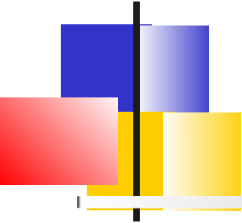
Garanties de livraison

- at-least-once : message jamais perdu, retraitable → idempotence.
- Commit throttled (défaut) ou latest → at-least-once.
- Commit ignore : auto-commit Kafka → aucune garantie.
- Erreur fail (défaut) : flux stoppé ; dead-letter-queue : isole et continue.



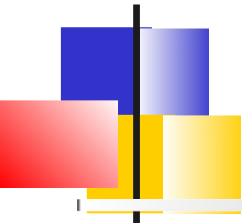
Messaging

Smallrye Reactive Messaging
Connecteur Kafka
Ack, Erreurs, Stratégies de commit
Tests



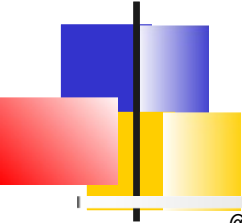
Approches de test

- Connecteur in-memory : remplace Kafka par un connecteur en mémoire. Tests unitaires rapides, sans broker.
- Dev Services : un vrai broker démarre automatiquement pour le profil test, sans configuration.
- Kafka Companion : pilote un vrai broker depuis le test (produire / consommer / vérifier).
- Règle : in-memory pour la logique métier, Dev Services / Companion pour l'intégration réelle.



Connecteur in-memory

- Dépendance de test : `smallrye-reactive-messaging-in-memory`.
- Basculer les canaux en mémoire dans un `QuarkusTestResourceLifecycleManager`.
- `switchIncomingChannelsToInMemory(...)` / `switchOutgoingChannelsToInMemory(...)`.
- `InMemoryConnector.clear()` en fin de test pour réinitialiser.



Test in-memory

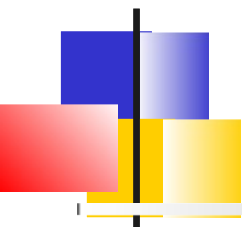
```
@QuarkusTest
@WithTestResource(InMemoryLifecycleManager.class)
class PriceProcessorTest {

    @Inject @Any InMemoryConnector connector;

    @Test
    void convertit_le_prix() {
        InMemorySource<Double> in = connector.source("prices");
        InMemorySink<Double> out = connector.sink("prices-out");

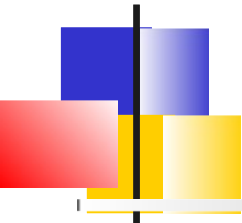
        in.send(100.0);

        assertEquals(1, out.received().size());
    }
}
```



Dev Services & Companion

- Dev Services : un broker réel démarre pour le profil test ; le test parle au vrai connecteur Kafka.
- Kafka Companion : dépendance quarkus-test-kafka-companion.
- `@InjectKafkaCompanion KafkaCompanion companion;`
- Produire / consommer des records pour vérifier de bout en bout (clés, sérialisation, partitions).



Bonnes pratiques de test

- Tester la logique avec le connecteur in-memory : rapide et déterministe.
- Gérer l'asynchrone avec Awaitility (`await().untilAsserted(...)`) plutôt que `Thread.sleep`.
- Isoler la configuration de test dans un `@TestProfile` ou un test resource dédié.
- Réserver Dev Services / Companion à la validation d'intégration (sérialisation, clés, partitionnement).